

SQL Postgres Mongo DB

Explain SQL
 DDL (CREATE, ALTER, DROP, TRUNCATE)
 DML (INSERT, UPDATE, DELETE)
 DQL (SELECT)
 TCL (COMMIT, ROLLBACK)
 DCL (GRANT, REVOKE)
 Interview Questions asked

[Show more](#)
[Show less](#)

SQL (Structured Query Language) - Interview Notes

SQL is a language used to communicate with relational databases such as MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

It is used to:

- Store data
- Retrieve data
- Update data
- Delete data
- Manage database objects
- Control access permissions

SQL Command Categories

1. DDL (Data Definition Language)

Used to define and modify database structure.

CREATE

Creates a database object.

```

SQL
CREATE TABLE Employee (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  salary DECIMAL(10,2)
);
  
```

ALTER

Modifies an existing object.

```
SQL
ALTER TABLE Employee
ADD email VARCHAR(100);
```

DROP

Deletes an object permanently.

```
SQL
DROP TABLE Employee;
```

TRUNCATE

Removes all rows but keeps table structure.

```
SQL
TRUNCATE TABLE Employee;
```

DDL Interview Questions

Q1. Difference between DELETE, TRUNCATE, and DROP?

Feature	DELETE	TRUNCATE	DROP
Removes Data	Yes	Yes	Yes
Removes Table Structure	No	No	Yes
WHERE Clause	Yes	No	No
Rollback Possible	Yes*	Depends on DB	No
Resets Identity	No	Yes	N/A

Example Interview Answer:

- DELETE removes selected rows.
- TRUNCATE removes all rows quickly.
- DROP removes both data and table structure.

Q2. Can we rollback TRUNCATE?

Answer:

- Oracle: No
- SQL Server/PostgreSQL: Usually yes inside transaction

Depends on database implementation.

2. DML (Data Manipulation Language)

Used to manipulate data.

INSERT

```
SQL
INSERT INTO Employee(id,name,salary)
VALUES(1, 'John', 50000);
```

UPDATE

```
SQL
UPDATE Employee
SET salary=60000
WHERE id=1;
```

DELETE

```
SQL
DELETE FROM Employee
WHERE id=1;
```

DML Interview Questions

Q1. What happens if UPDATE has no WHERE clause?

```
SQL
UPDATE Employee
SET salary=10000;
```

All records get updated.

This is a very common interview question.

Q2. Difference between DELETE and TRUNCATE?

DELETE

```
SQL
DELETE FROM Employee
WHERE id=1;
```

TRUNCATE

```
SQL
TRUNCATE TABLE Employee;
```

DELETE is row-by-row and can use WHERE.

TRUNCATE removes all rows at once.

3. DQL (Data Query Language)

Used to retrieve data.

SELECT

```
SQL
SELECT * FROM Employee;
```

Select Specific Columns

```
SQL
SELECT id, name
FROM Employee;
```

WHERE

```
SQL
SELECT *
FROM Employee
WHERE salary > 50000;
```

ORDER BY

```
SQL
SELECT *
FROM Employee
ORDER BY salary DESC;
```

GROUP BY

```
SQL
SELECT department,
COUNT(*)
FROM Employee
GROUP BY department;
```

HAVING

```
SQL
SELECT department,
COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*) > 5;
```

DQL Interview Questions

Q1. Execution Order of SQL Query?

```
SQL
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

Actual execution:

```
FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
LIMIT
```

Very frequently asked.

Q2. Difference between WHERE and HAVING?

WHERE	HAVING
Filters rows	Filters groups
Before GROUP BY	After GROUP BY
Cannot use aggregate	Can use aggregate

Example:

```
SQL
SELECT dept,
COUNT(*)
FROM Employee
GROUP BY dept
HAVING COUNT(*) > 5;
```

4. TCL (Transaction Control Language)

Used to manage transactions.

COMMIT

Saves changes permanently.

```
SQL
COMMIT;
```

ROLLBACK

Undo changes.

```
SQL
ROLLBACK;
```

SAVEPOINT

Creates a checkpoint.

```
SQL
SAVEPOINT sp1;
```

Example

```
SQL
BEGIN;

UPDATE Account
SET balance=balance-1000
WHERE id=1;

UPDATE Account
SET balance=balance+1000
WHERE id=2;

COMMIT;
```

TCL Interview Questions

Q1. What is a Transaction?

A transaction is a group of SQL statements executed as a single unit.

Example:

Money transfer between two accounts.

Q2. What are ACID Properties?

A - Atomicity

All operations succeed or none.

C - Consistency

Database remains valid.

I - Isolation

Transactions don't interfere.

D - Durability

Committed data survives crash.

This is one of the most asked SQL interview questions.

5. DCL (Data Control Language)

Used to manage permissions.

GRANT

Give access.

```
SQL
GRANT SELECT, INSERT
ON Employee
TO user1;
```

REVOKE

Remove access.

```
SQL
REVOKE INSERT
ON Employee
FROM user1;
```

DCL Interview Questions

Q1. Difference between GRANT and REVOKE?

GRANT	REVOKE
Gives Permission	Removes Permission

Frequently Asked SQL Interview Questions (2 Years Experience)

Basic

1. What is SQL?
2. What is RDBMS?
3. Difference between SQL and MySQL?
4. What is Primary Key?
5. What is Foreign Key?
6. What is Unique Key?
7. What is Composite Key?
8. What is Candidate Key?
9. What is Surrogate Key?

Intermediate

1. Difference between DELETE, DROP, TRUNCATE?
2. Difference between WHERE and HAVING?
3. Difference between UNION and UNION ALL?
4. Difference between INNER JOIN and OUTER JOIN?
5. Difference between CHAR and VARCHAR?
6. Difference between Primary Key and Unique Key?
7. What is Normalization?
8. What is Denormalization?
9. What are ACID properties?
10. What is Indexing?

Query-Based Questions

Find Second Highest Salary

```
SQL
SELECT MAX(salary)
FROM Employee
WHERE salary <
(
    SELECT MAX(salary)
    FROM Employee
);
```

Find Duplicate Records

```
SQL
SELECT email,
       COUNT(*)
FROM Employee
GROUP BY email
HAVING COUNT(*) > 1;
```

Find Employees Without Department

```
SQL
SELECT *
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.id
WHERE d.id IS NULL;
```

Scenario-Based Questions

Banking Example

What happens if money is deducted from Account A but not credited to Account B?

Answer:

Rollback occurs because transaction must maintain Atomicity.

Production Issue

Someone executed:

```
SQL
UPDATE Employee
SET salary = 0;
```

without WHERE clause.

What will you do?

Answer:

- Check if transaction committed.
 - Use ROLLBACK if possible.
 - Restore from backup if committed.
-

For a Java Full Stack Developer interview with 2 years experience, after these basics, the next most important topics are:

1. Joins
2. Subqueries
3. Indexes
4. Normalization
5. Views
6. Stored Procedures
7. Transactions & ACID
8. Window Functions (ROW_NUMBER, RANK, DENSE_RANK)
9. CTEs
10. Query Optimization

These topics are asked very frequently in companies like Tata Consultancy Services, Accenture, Tech Mahindra, and Vertoz.

```
WHERE
DISTINCT
ORDER BY
LIMIT
OFFSET
BETWEEN
```

IN
LIKE
Show moreShow less

SQL Clauses and Operators (Interview Notes)

These are some of the most commonly asked SQL concepts in interviews.

1. WHERE Clause

Used to filter records based on a condition.

Syntax

```
SQL
SELECT *
FROM Employee
WHERE salary > 50000;
```

Example

Employee Table

id	name	salary
1	John	40000
2	Alice	60000
3	Bob	70000

Query:

```
SQL
SELECT *
FROM Employee
WHERE salary > 50000;
```

Output:

id	name	salary
2	Alice	60000
3	Bob	70000

Interview Questions

Q. Can WHERE be used with aggregate functions?

✗ No

```
SQL
SELECT *
FROM Employee
WHERE COUNT(*) > 5;
```

Invalid

Use HAVING instead.

2. DISTINCT

Removes duplicate values.

Syntax

```
SQL
SELECT DISTINCT department
FROM Employee;
```

Example

Data

department
IT
HR
IT
Finance

Query

```
SQL
SELECT DISTINCT department
FROM Employee;
```

Output

department
IT
HR
Finance

Interview Questions

Q. Difference between DISTINCT and GROUP BY?

DISTINCT	GROUP BY
Removes duplicates	Groups records
Simpler	Used with aggregate functions

Example:

```
SQL
SELECT DISTINCT department
FROM Employee;
```

```
SQL
SELECT department
FROM Employee
GROUP BY department;
```

Both return same result.

3. ORDER BY

Sorts data.

Ascending (Default)

```
SQL
SELECT *
FROM Employee
ORDER BY salary;
```

Descending

```
SQL
SELECT *
FROM Employee
ORDER BY salary DESC;
```

Multiple Columns

```
SQL
SELECT *
FROM Employee
ORDER BY department ASC,
salary DESC;
```

Interview Questions

Q. What is default sorting order?

☒ ASC (Ascending)

```
SQL
ORDER BY salary
```

Same as:

```
SQL
ORDER BY salary ASC
```

4. LIMIT

Restricts number of rows returned.

Syntax

```
SQL
SELECT *
FROM Employee
LIMIT 5;
```

Returns first 5 records.

Example

```
SQL
SELECT *
FROM Employee
ORDER BY salary DESC
LIMIT 3;
```

Top 3 highest salaries.

Interview Questions

Q. SQL Server equivalent of LIMIT?

```
SQL
SELECT TOP 5 *
FROM Employee;
```

Or

```
SQL
SELECT *
FROM Employee
OFFSET 0 ROWS
FETCH NEXT 5 ROWS ONLY;
```

5. OFFSET

Skips rows before returning data.

Syntax

```
SQL
SELECT *
FROM Employee
LIMIT 5 OFFSET 10;
```

Skip first 10 rows and return next 5.

Pagination Example

Page Size = 10

Page 1

```
SQL
SELECT *
FROM Employee
LIMIT 10 OFFSET 0;
```

Page 2

```
SQL
SELECT *
FROM Employee
LIMIT 10 OFFSET 10;
```

Page 3

```
SQL
SELECT *
FROM Employee
LIMIT 10 OFFSET 20;
```

Formula:

```
OFFSET = (Page Number - 1) * Page Size
```

Interview Questions

Q. How is pagination implemented in SQL?

```
SQL
SELECT *
FROM Employee
ORDER BY id
LIMIT 10 OFFSET 20;
```

Used in web applications.

6. BETWEEN

Checks whether value lies within a range.

Syntax

```
SQL
SELECT *
FROM Employee
WHERE salary BETWEEN 30000 AND 50000;
```

Equivalent to:

```
SQL
SELECT *
FROM Employee
WHERE salary >= 30000
AND salary <= 50000;
```

Date Example

```
SQL
SELECT *
FROM Orders
WHERE order_date
BETWEEN '2026-01-01'
AND '2026-12-31';
```

Interview Questions

Q. Is BETWEEN inclusive?

☒ Yes

```
SQL
BETWEEN 10 AND 20
```

Includes:

```
10
20
```

7. IN Operator

Used to match multiple values.

Syntax

```
SQL
SELECT *
FROM Employee
WHERE department IN ('IT', 'HR');
```

Equivalent to:

```
SQL
SELECT *
FROM Employee
```

```
WHERE department='IT'
OR department='HR';
```

Example

```
SQL
SELECT *
FROM Employee
WHERE id IN (1,2,3);
```

Subquery Example

```
SQL
SELECT *
FROM Employee
WHERE dept_id IN
(
    SELECT id
    FROM Department
);
```

Interview Questions

Q. IN vs EXISTS?

IN	EXISTS
Compares values	Checks existence
Better for small datasets	Better for large datasets
Loads all values	Stops at first match

8. LIKE Operator

Used for pattern matching.

Wildcards

Symbol	Meaning
%	Any number of characters
_	Exactly one character

Starts With

```
SQL
SELECT *
FROM Employee
WHERE name LIKE 'A%';
```

Output:

Alice
Andrew
Alex

Ends With

```
SQL
SELECT *
FROM Employee
WHERE name LIKE '%n';
```

Output:

John
Robin

Contains

```
SQL
SELECT *
FROM Employee
WHERE name LIKE '%lex%';
```

Output:

Alex
Alexander

Single Character

```
SQL
SELECT *
FROM Employee
WHERE name LIKE 'J_n';
```

Matches:

Jan
Jen
Jon

Not:

John

Interview Questions

Q. Difference between % and _ ?

```
SQL
'A%'
```

Any length after A.

Examples:

```
A
Alex
Andrew
```

```
SQL
```

```
'A_'
```

Exactly one character after A.

Examples:

```
AB
AX
```

Not:

```
Alex
```

Frequently Asked Interview Questions

Q1. Logical Execution Order of SQL Query

```
FROM
WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY
LIMIT/OFFSET
```

Very Frequently Asked.

Q2. Fetch Top 5 Highest Salaries

```
SQL
SELECT *
FROM Employee
ORDER BY salary DESC
LIMIT 5;
```

Q3. Find Employees Whose Name Starts With 'A'

```
SQL
SELECT *
FROM Employee
WHERE name LIKE 'A%';
```

Q4. Find Employees in IT or HR Department

```
SQL
```

```
SELECT *
FROM Employee
WHERE department IN ('IT', 'HR');
```

Q5. Get Employees With Salary Between 50k and 100k

```
SQL
SELECT *
FROM Employee
WHERE salary BETWEEN 50000 AND 100000;
```

Q6. Remove Duplicate Departments

```
SQL
SELECT DISTINCT department
FROM Employee;
```

One-Line Interview Summary

Clause	Purpose
WHERE	Filter rows
DISTINCT	Remove duplicates
ORDER BY	Sort records
LIMIT	Restrict rows
OFFSET	Skip rows
BETWEEN	Range filtering
IN	Match multiple values
LIKE	Pattern matching

These clauses are used in almost every SQL interview coding question and are essential before moving on to **JOINS**, **SUBQUERIES**, **INDEXES**, and **WINDOW FUNCTIONS**.

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

SQL Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single result.

Most commonly asked aggregate functions:

1. COUNT()
2. SUM()
3. AVG()

4. MIN()

5. MAX()

Sample Employee Table

id	name	department	salary
1	John	IT	50000
2	Alice	IT	70000
3	Bob	HR	40000
4	David	HR	60000
5	Emma	Finance	80000

1. COUNT()

Used to count rows.

Count All Rows

SQL

```
SELECT COUNT(*)
FROM Employee;
```

Output:

5

Count Non-NULL Values

SQL

```
SELECT COUNT(salary)
FROM Employee;
```

Counts only non-null salary values.

Count Distinct Values

SQL

```
SELECT COUNT(DISTINCT department)
FROM Employee;
```

Output:

3

(IT, HR, Finance)

Interview Questions

Difference between COUNT(*) and COUNT(column)?

SQL
COUNT(*)

Counts all rows.

SQL
COUNT(salary)

Counts only rows where salary is NOT NULL.

2. SUM()

Calculates total value.

Total Salary

SQL
SELECT SUM(salary)
FROM Employee;

Output:

300000

Department Wise Salary

SQL
SELECT department,
SUM(salary)
FROM Employee
GROUP BY department;

Output:

department	sum
IT	120000
HR	100000
Finance	80000

Interview Questions

Can SUM() work on VARCHAR?

✗ No

SUM works only on numeric columns.

3. AVG()

Calculates average value.

Average Salary

```
SQL
SELECT AVG(salary)
FROM Employee;
```

Output:

60000

Department Wise Average Salary

```
SQL
SELECT department,
AVG(salary)
FROM Employee
GROUP BY department;
```

Interview Questions

Does AVG ignore NULL values?

✓ Yes

Example:

salary
10000
20000
NULL

```
SQL
SELECT AVG(salary)
FROM Employee;
```

Result:

15000

NULL is ignored.

4. MIN()

Returns smallest value.

Lowest Salary

```
SQL
SELECT MIN(salary)
FROM Employee;
```

Output:

```
40000
```

Earliest Joining Date

```
SQL
SELECT MIN(join_date)
FROM Employee;
```

Returns oldest date.

Interview Questions

Can MIN work on Strings?

☒ Yes

```
SQL
SELECT MIN(name)
FROM Employee;
```

Returns alphabetically first name.

5. MAX()

Returns largest value.

Highest Salary

```
SQL
SELECT MAX(salary)
FROM Employee;
```

Output:

80000

Latest Date

SQL

```
SELECT MAX(join_date)
FROM Employee;
```

Returns most recent date.

Interview Questions

Can MAX work on Strings?

☒ Yes

SQL

```
SELECT MAX(name)
FROM Employee;
```

Returns alphabetically last value.

Aggregate Functions with GROUP BY

Department Wise Employee Count

SQL

```
SELECT department,
COUNT(*) AS employee_count
FROM Employee
GROUP BY department;
```

Output:

department	employee_count
IT	2
HR	2
Finance	1

Department Wise Statistics

SQL

```
SELECT department,
COUNT(*) AS employees,
SUM(salary) AS total_salary,
AVG(salary) AS avg_salary,
MIN(salary) AS min_salary,
```



```
MAX(salary) AS max_salary
FROM Employee
GROUP BY department;
```

This is a very common interview question.

Aggregate Functions with HAVING

Departments Having More Than 2 Employees

```
SQL
SELECT department,
COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*) > 2;
```

Departments With Average Salary > 50000

```
SQL
SELECT department,
AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 50000;
```

Frequently Asked Interview Questions

Q1. Difference Between WHERE and HAVING?

WHERE	HAVING
Filters rows	Filters groups
Before GROUP BY	After GROUP BY
Cannot use aggregate functions	Can use aggregate functions

Example:

```
SQL
SELECT department,
COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*) > 1;
```

Q2. Can Aggregate Functions Ignore NULL Values?

Function	Ignores NULL?
COUNT(column)	Yes
SUM()	Yes
AVG()	Yes

Function	Ignores NULL?
MIN()	Yes
MAX()	Yes

Q3. Can We Use Aggregate Functions Without GROUP BY?

✓ Yes

```
SQL
SELECT AVG(salary)
FROM Employee;
```

Returns a single result.

Q4. What Happens If You Use Non-Aggregated Columns Without GROUP BY?

✗ Error

```
SQL
SELECT department,
      AVG(salary)
FROM Employee;
```

Most databases will throw an error because **department** is neither aggregated nor grouped.

Correct:

```
SQL
SELECT department,
      AVG(salary)
FROM Employee
GROUP BY department;
```

Q5. Find the Second Highest Salary

```
SQL
SELECT MAX(salary)
FROM Employee
WHERE salary <
(
  SELECT MAX(salary)
  FROM Employee
);
```

One-Line Interview Summary

Function	Purpose
COUNT()	Count rows
SUM()	Total value

Function	Purpose
AVG()	Average value
MIN()	Smallest value
MAX()	Largest value

Common Interview Query

SQL

```
SELECT department,
COUNT(*) AS total_employees,
SUM(salary) AS total_salary,
AVG(salary) AS avg_salary,
MIN(salary) AS lowest_salary,
MAX(salary) AS highest_salary
FROM Employee
GROUP BY department
HAVING AVG(salary) > 50000;
```

This single query demonstrates **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**, **GROUP BY**, and **HAVING** together—an interviewer favorite for Java Full Stack and Backend Developer roles.

GROUP BY
HAVING
Explain when to use how to use

GROUP BY and HAVING – When to Use and How to Use

These are among the most frequently asked SQL interview topics because they are used for reporting, analytics, dashboards, and data aggregation.

1. GROUP BY

What is GROUP BY?

GROUP BY groups rows having the same value into a single group.

It is generally used with aggregate functions such as:

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

Employee Table

id	name	department	salary
1	John	IT	50000
2	Alice	IT	70000
3	Bob	HR	40000
4	David	HR	60000
5	Emma	Finance	80000

Without GROUP BY

```
SQL
SELECT AVG(salary)
FROM Employee;
```

Output:

```
60000
```

Average salary of all employees.

With GROUP BY

```
SQL
SELECT department,
AVG(salary)
FROM Employee
GROUP BY department;
```

Output:

department	avg_salary
IT	60000
HR	50000
Finance	80000

What happened?

SQL created 3 groups:

```
IT      -> John, Alice
HR      -> Bob, David
Finance -> Emma
```

Then AVG() was calculated for each group.

When Should We Use GROUP BY?

Use GROUP BY when you need:

Count Employees Department Wise

```
SQL
SELECT department,
       COUNT(*)
FROM Employee
GROUP BY department;
```

Total Salary Department Wise

```
SQL
SELECT department,
       SUM(salary)
FROM Employee
GROUP BY department;
```

Average Salary Department Wise

```
SQL
SELECT department,
       AVG(salary)
FROM Employee
GROUP BY department;
```

Highest Salary Department Wise

```
SQL
SELECT department,
       MAX(salary)
FROM Employee
GROUP BY department;
```

Real-World Examples

E-commerce

Find total sales per category

```
SQL
SELECT category,
       SUM(amount)
FROM Orders
GROUP BY category;
```

Banking

Find number of accounts per branch

```
SQL
SELECT branch_id,
       COUNT(*)
FROM Accounts
GROUP BY branch_id;
```

HR System

Find average salary per department

```
SQL
SELECT department,
       AVG(salary)
FROM Employee
GROUP BY department;
```

Interview Question

Why is GROUP BY needed?

Without GROUP BY:

```
SQL
SELECT department,
       COUNT(*)
FROM Employee;
```

✗ Error

Because SQL doesn't know which department value to display.

Correct:

```
SQL
SELECT department,
       COUNT(*)
FROM Employee
GROUP BY department;
```

2. HAVING

What is HAVING?

HAVING filters groups created by GROUP BY.

Think:

```
WHERE -> Filters rows
HAVING -> Filters groups
```

Example

Find departments having more than 1 employee.

```
SQL
SELECT department,
       COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*) > 1;
```

Output:

department
IT
HR

Finance is removed because it has only one employee.

Execution Flow

```
SQL
SELECT department,
       COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*) > 1;
```

Step 1

GROUP BY creates groups

```
IT      -> 2 employees
HR      -> 2 employees
Finance -> 1 employee
```

Step 2

HAVING applies condition

```
COUNT(*) > 1
```

Result:

```
IT
HR
```

When Should We Use HAVING?

Use HAVING when filtering aggregate results.

Departments with Average Salary > 50000

```
SQL
SELECT department,
       AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 50000;
```

Output:

department
IT
Finance

Departments with Total Salary > 100000

```
SQL
SELECT department,
       SUM(salary)
FROM Employee
GROUP BY department
HAVING SUM(salary) > 100000;
```

Departments with More Than 5 Employees

```
SQL
SELECT department,
       COUNT(*)
FROM Employee
GROUP BY department
HAVING COUNT(*) > 5;
```

WHERE vs HAVING

This is asked in almost every SQL interview.

WHERE	HAVING
Filters rows	Filters groups
Before GROUP BY	After GROUP BY
Cannot use aggregate functions	Can use aggregate functions
Faster	Slightly slower

WHERE Example

```
SQL
```



```
SELECT *
FROM Employee
WHERE salary > 50000;
```

Filters individual employees.

HAVING Example

```
SQL
SELECT department,
       AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 50000;
```

Filters departments.

Using WHERE and HAVING Together

Find departments with average salary > 50000, considering only employees earning > 40000.

```
SQL
SELECT department,
       AVG(salary)
FROM Employee
WHERE salary > 40000
GROUP BY department
HAVING AVG(salary) > 50000;
```

Execution

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Common Interview Questions

Q1. Can HAVING be used without GROUP BY?

☒ Yes

```
SQL
SELECT COUNT(*)
FROM Employee
HAVING COUNT(*) > 5;
```

Though it's uncommon.

Q2. Can WHERE use COUNT()?

✗ No

```
SQL
SELECT *
FROM Employee
WHERE COUNT(*) > 1;
```

Invalid.

Use HAVING.

Q3. Which executes first: WHERE or HAVING?

✓ WHERE

Execution order:

```
FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
LIMIT
```

Q4. Why is WHERE faster than HAVING?

Because:

- WHERE filters rows before grouping.
- HAVING filters after grouping.

Less data is processed.

Interview Shortcut

Use WHERE when filtering individual rows

```
SQL
WHERE salary > 50000
```

Use GROUP BY when creating groups

```
SQL
GROUP BY department
```

Use HAVING when filtering aggregated results

```
SQL
```

HAVING AVG(salary) > 50000

Most Common Interview Query

```
SQL
SELECT department,
       COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary
FROM Employee
WHERE salary > 30000
GROUP BY department
HAVING COUNT(*) > 1
ORDER BY avg_salary DESC;
```

This single query demonstrates:

- WHERE
- GROUP BY
- COUNT
- AVG
- HAVING
- ORDER BY

and is a classic SQL interview question for 2–3 years experienced Java/Full Stack developers.

INNER JOIN
LEFT JOIN
RIGHT JOIN
FULL JOIN
SELF JOIN
CROSS JOIN

SQL JOINS - Complete Interview Guide

A JOIN is used to combine rows from two or more tables based on a related column.

Sample Tables

Employee

emp_id	name	dept_id
1	John	101
2	Alice	102
3	Bob	103
4	David	NULL

Department

dept_id	dept_name
101	IT
102	HR
104	Finance

1. INNER JOIN

What is INNER JOIN?

Returns only matching records from both tables.

Syntax

```
SQL
SELECT e.emp_id,
       e.name,
       d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

Result

emp_id	name	dept_name
1	John	IT
2	Alice	HR

Visualization

Employee n Department

Only common records.

When to Use?

When you need records existing in both tables.

Example

- Employees with departments
- Orders with customers

- Students with courses

Interview Question

Difference Between JOIN and INNER JOIN?

```
SQL
SELECT *
FROM A
JOIN B ON ...
```

and

```
SQL
SELECT *
FROM A
INNER JOIN B ON ...
```

Both are the same.

2. LEFT JOIN

What is LEFT JOIN?

Returns:

- All rows from left table
- Matching rows from right table
- NULL if no match

Syntax

```
SQL
SELECT e.emp_id,
       e.name,
       d.dept_name
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id;
```

Result

emp_id	name	dept_name
1	John	IT
2	Alice	HR
3	Bob	NULL
4	David	NULL

Visualization

Entire Employee table
+
Matching Department rows

When to Use?

Show all employees even if they don't have departments.

Interview Question

Find Employees Without Department

```
SQL
SELECT *
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Output:

Bob
David

Very frequently asked.

3. RIGHT JOIN

What is RIGHT JOIN?

Returns:

- All rows from right table
- Matching rows from left table
- NULL if no match

Syntax

```
SQL
SELECT e.name,
       d.dept_name
FROM Employee e
```

```
RIGHT JOIN Department d
ON e.dept_id = d.dept_id;
```

Result

name	dept_name
John	IT
Alice	HR
NULL	Finance

Finance exists even without employees.

Visualization

Entire Department table
+
Matching Employee rows

When to Use?

Show all departments even if no employees exist.

Interview Tip

Most developers prefer LEFT JOIN by swapping table positions.

4. FULL JOIN (FULL OUTER JOIN)

What is FULL JOIN?

Returns:

- All rows from left table
- All rows from right table
- Matching rows merged

Syntax

```
SQL
SELECT *
FROM Employee e
FULL JOIN Department d
ON e.dept_id = d.dept_id;
```

Result

Employee	Department
John	IT
Alice	HR
Bob	NULL
David	NULL
NULL	Finance

Visualization

LEFT JOIN
+
RIGHT JOIN

Everything from both tables.

When to Use?

Data reconciliation and audits.

Example:

- Compare records between systems
- Find missing data

Interview Question

MySQL Support?

Older MySQL versions don't support FULL JOIN directly.

Use:

SQL
LEFT JOIN
UNION
RIGHT JOIN

5. SELF JOIN

What is SELF JOIN?

A table joins with itself.

Employee Table

emp_id	name	manager_id
1	CEO	NULL
2	John	1
3	Alice	1
4	Bob	2

Query

```
SQL
SELECT e.name AS Employee,
       m.name AS Manager
FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
```

Result

Employee	Manager
CEO	NULL
John	CEO
Alice	CEO
Bob	John

When to Use?

Hierarchical relationships:

- Employee → Manager
- Category → Parent Category
- Comments → Parent Comments

Interview Question

Why Self Join?

To compare rows within the same table.

6. CROSS JOIN

What is CROSS JOIN?

Returns Cartesian Product.

Every row from first table combines with every row from second table.

Table A

Color
Red
Blue

Table B

Size
S
M

Query

```
SQL
SELECT *
FROM Colors
CROSS JOIN Sizes;
```

Result

Color	Size
Red	S
Red	M
Blue	S
Blue	M

$2 \times 2 = 4$ rows

Formula

```
Rows Returned =
Rows(Table A) × Rows(Table B)
```

When to Use?

Generate combinations.

Examples:

- Product variants
- Seating arrangements
- Test data generation

Interview Question

Why Is CROSS JOIN Dangerous?

If:

Table A = 1,000 rows
Table B = 10,000 rows

Result:

10,000,000 rows

Huge performance issue.

Visual Summary

INNER JOIN
A $\cap$ B

LEFT JOIN
All A + Matching B

RIGHT JOIN
All B + Matching A

FULL JOIN
All A + All B

SELF JOIN
A joined with A

CROSS JOIN
A $\times$ B

Most Asked Interview Questions

Q1. Difference Between INNER and LEFT JOIN?

INNER JOIN	LEFT JOIN
Only matching records	All left records

INNER JOIN	LEFT JOIN
Unmatched rows ignored	Unmatched rows shown as NULL

Q2. Find Employees Without Department

```
SQL
SELECT e.*
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Q3. Which Join Returns All Records?

FULL JOIN

Q4. Which Join Creates Cartesian Product?

CROSS JOIN

Q5. Can a Table Join Itself?

☒ Yes

```
SQL
SELF JOIN
```

Interview Shortcut

Join	Returns
INNER JOIN	Matching rows only
LEFT JOIN	All left + matching right
RIGHT JOIN	All right + matching left
FULL JOIN	All rows from both tables
SELF JOIN	Table joined with itself
CROSS JOIN	Every combination of rows

2-Year Java Developer Interview Favorite

```
SQL
SELECT e.name,
       d.dept_name
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Question: "Find employees who are not assigned to any department."

This is one of the most commonly asked SQL JOIN questions in interviews.

Explain subquery
Correlated Subquery
Nested Subquery

SQL Subqueries – Complete Interview Guide

A **Subquery** is a query written inside another query.

It is also called an **Inner Query** or **Nested Query**.

```
SQL
SELECT *
FROM Employee
WHERE salary >
(
    SELECT AVG(salary)
    FROM Employee
);
```

The inner query executes first, and its result is used by the outer query.

Types of Subqueries

1. Single Row Subquery
2. Multiple Row Subquery
3. Nested Subquery
4. Correlated Subquery

Sample Tables

Employee

emp_id	name	dept_id	salary
1	John	101	50000
2	Alice	101	70000
3	Bob	102	40000
4	David	102	60000
5	Emma	103	80000

Department

dept_id	dept_name
101	IT
102	HR
103	Finance

1. Nested Subquery

What is a Nested Subquery?

A subquery inside another query.

The inner query executes first.

Example 1: Employees Earning More Than Average Salary

```
SQL
SELECT *
FROM Employee
WHERE salary >
(
    SELECT AVG(salary)
    FROM Employee
);
```

Execution

Step 1

```
SQL
SELECT AVG(salary)
FROM Employee;
```

Result:

```
60000
```

Step 2

Outer query becomes:

```
SQL
SELECT *
FROM Employee
WHERE salary > 60000;
```

Output:

name	salary
Alice	70000

name	salary
Emma	80000

Example 2: Employees Working in IT Department

```
SQL
SELECT *
FROM Employee
WHERE dept_id =
(
    SELECT dept_id
    FROM Department
    WHERE dept_name='IT'
);
```

Interview Question

Find Employee with Highest Salary

```
SQL
SELECT *
FROM Employee
WHERE salary =
(
    SELECT MAX(salary)
    FROM Employee
);
```

Nested Subquery with IN

Example

Find employees belonging to IT and HR departments.

```
SQL
SELECT *
FROM Employee
WHERE dept_id IN
(
    SELECT dept_id
    FROM Department
    WHERE dept_name IN ('IT', 'HR')
);
```

Multi-Level Nested Subquery

Subquery inside another subquery.

```
SQL
SELECT *
FROM Employee
```

```
WHERE dept_id =
(
  SELECT dept_id
  FROM Department
  WHERE dept_name =
    (
      SELECT 'IT'
    )
);
```

Not common in real projects.

2. Correlated Subquery

What is a Correlated Subquery?

A correlated subquery depends on the outer query.

It executes once for every row of the outer query.

Key Point

Nested Subquery:
Runs once.

Correlated Subquery:
Runs for each row.

Example

Find employees earning more than the average salary of their department.

```
SQL
SELECT e1.*
FROM Employee e1
WHERE salary >
(
  SELECT AVG(e2.salary)
  FROM Employee e2
  WHERE e2.dept_id = e1.dept_id
);
```

How It Works

For John (IT)

```
SQL
SELECT AVG(salary)
FROM Employee
WHERE dept_id=101;
```

Average:

```
60000
```


John salary:

50000

Not selected.

For Alice (IT)

Average:

60000

Salary:

70000

Selected.

For Bob (HR)

Average:

50000

Salary:

40000

Not selected.

For David (HR)

Average:

50000

Salary:

60000

Selected.

Output

name
Alice
David

Why Correlated Subquery Is Slow

Because it executes repeatedly.

Example:

1000 Employees

Subquery executes:

1000 Times

Can impact performance.

Correlated Subquery vs Nested Subquery

Feature	Nested Subquery	Correlated Subquery
Depends on outer query	No	Yes
Execution count	Once	For every row
Performance	Faster	Slower
Uses outer table alias	No	Yes

Nested Subquery Example

```
SQL
SELECT *
FROM Employee
WHERE salary >
(
    SELECT AVG(salary)
    FROM Employee
);
```

Runs once.

Correlated Subquery Example

```
SQL
SELECT *
FROM Employee e1
WHERE salary >
(
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e1.dept_id=e2.dept_id
);
```

Runs for every employee.

EXISTS with Correlated Subquery

Very common interview question.

Find Departments Having Employees

```
SQL
SELECT *
FROM Department d
WHERE EXISTS
(
    SELECT 1
    FROM Employee e
    WHERE e.dept_id=d.dept_id
);
```

Explanation

For each department:

Check whether employee exists.

If yes:

Return department.

IN vs EXISTS

IN

```
SQL
SELECT *
FROM Employee
WHERE dept_id IN
(
    SELECT dept_id
    FROM Department
);
```

EXISTS

```
SQL
SELECT *
FROM Employee e
WHERE EXISTS
(
    SELECT 1
    FROM Department d
    WHERE d.dept_id=e.dept_id
);
```

Interview Answer

IN	EXISTS
Compares values	Checks existence
Better for small datasets	Better for large datasets
Evaluates complete result	Stops at first match

Most Asked Interview Questions

Q1. What is a Subquery?

A query inside another query.

Q2. What is a Correlated Subquery?

A subquery that references columns from the outer query.

Q3. Which Is Faster?

☒ Nested Subquery

Because it executes only once.

Q4. Difference Between JOIN and Subquery?

JOIN	Subquery
Combines tables	Query inside query
Usually faster	Sometimes slower
Easier for large data	Easier for simple logic

Q5. Find Employees Earning More Than Average Salary

```
SQL
SELECT *
FROM Employee
WHERE salary >
(
    SELECT AVG(salary)
    FROM Employee
);
```

Q6. Find Highest Salary Employee

```
SQL
```

```
SELECT *
FROM Employee
WHERE salary =
(
    SELECT MAX(salary)
    FROM Employee
);
```

Q7. Find Employees Earning More Than Department Average

```
SQL
SELECT e1.*
FROM Employee e1
WHERE salary >
(
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id=e1.dept_id
);
```

This is one of the most common **2-5 years experience SQL interview questions**.

Interview Shortcut

Nested Subquery

Inner query runs first.
Outer query uses result.
Runs once.

Correlated Subquery

References outer query.
Runs for each row.
Can be slower.

Quick Memory Trick

```
Nested Subquery
=
Independent

Correlated Subquery
=
Dependent
```

If the inner query can run by itself → **Nested Subquery**.

If the inner query needs a value from the outer query (`e1.dept_id`) → **Correlated Subquery**.

Constraints
Primary Key
Foreign Key
Unique Key
Check Constraint

Default Constraint

Show moreShow less

SQL Constraints – Complete Interview Guide

Constraints are rules applied on table columns to ensure **data integrity, accuracy, and consistency**. They prevent invalid data from being inserted into the database.

Why Constraints?

Without constraints:

```
Employee ID = NULL
Salary = -5000
Duplicate Email IDs
Invalid Department IDs
```

could be inserted into the database.

Constraints prevent such issues.

Types of Constraints

1. PRIMARY KEY
2. FOREIGN KEY
3. UNIQUE KEY
4. CHECK CONSTRAINT
5. DEFAULT CONSTRAINT
6. NOT NULL (also important for interviews)

Sample Table

```
SQL
CREATE TABLE Employee(
  emp_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  salary DECIMAL(10,2) CHECK (salary > 0),
  dept_id INT,
  created_date DATE DEFAULT CURRENT_DATE
);
```

1. PRIMARY KEY

What is a Primary Key?

A column (or set of columns) that uniquely identifies each row.

Rules

- Cannot contain NULL
- Cannot contain duplicate values
- Only one Primary Key per table

Example

```
SQL
CREATE TABLE Employee(
  emp_id INT PRIMARY KEY,
  name VARCHAR(100)
);
```

Valid Data

emp_id	name
1	John
2	Alice

Invalid Data

emp_id	name
1	John
1	Alice

✗ Duplicate Primary Key

Invalid Data

emp_id	name
NULL	John

✗ NULL not allowed

Interview Questions

Q1. Why do we need a Primary Key?

To uniquely identify each record.

Example:

```
Employee ID
Customer ID
Order ID
```

Q2. Can a Table Have Multiple Primary Keys?

✗ No

Only one Primary Key.

But it can be a combination of columns (Composite Key).

Composite Primary Key

```
SQL
CREATE TABLE Student_Course(
    student_id INT,
    course_id INT,
    PRIMARY KEY(student_id, course_id)
);
```

Combination must be unique.

2. FOREIGN KEY

What is a Foreign Key?

A Foreign Key creates a relationship between two tables.

It ensures referential integrity.

Department Table

```
SQL
CREATE TABLE Department(
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

Employee Table

```
SQL
CREATE TABLE Employee(
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    dept_id INT,
    FOREIGN KEY(dept_id)
    REFERENCES Department(dept_id)
);
```


Valid Insert

Department

dept_id
101
102

Employee

emp_id	dept_id
1	101

✓ Works

Invalid Insert

```
SQL
INSERT INTO Employee
VALUES (1, 'John', 999);
```

✗ Error

Because department 999 does not exist.

Interview Questions

Q1. Why Foreign Key?

To maintain relationships between tables.

Example:

```
Employee → Department
Order → Customer
Student → Course
```

Q2. Can Foreign Key Have NULL?

✓ Yes

```
SQL
INSERT INTO Employee
VALUES (1, 'John', NULL);
```

Valid unless NOT NULL is specified.

3. UNIQUE KEY

What is Unique Key?

Ensures all values in a column are unique.

Unlike Primary Key, NULL is allowed.

Example

```
SQL
CREATE TABLE Employee(
  emp_id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE
);
```

Valid Data

email
a@gmail.com
b@gmail.com

Invalid Data

email
a@gmail.com
a@gmail.com

✗ Duplicate value

Interview Questions

Primary Key vs Unique Key

Primary Key	Unique Key
No NULL	NULL allowed
One per table	Multiple allowed
Unique	Unique

Example

```
SQL
emp_id -> PRIMARY KEY
email -> UNIQUE
```

```
phone -> UNIQUE  
aadhaar -> UNIQUE
```

A table can have many UNIQUE constraints.

4. CHECK Constraint

What is CHECK?

Restricts values based on a condition.

Example

```
SQL  
CREATE TABLE Employee(  
    emp_id INT,  
    salary DECIMAL(10,2)  
    CHECK(salary > 0)  
);
```

Valid Insert

```
SQL  
INSERT INTO Employee  
VALUES(1,50000);
```

✓ Success

Invalid Insert

```
SQL  
INSERT INTO Employee  
VALUES(1,-5000);
```

✗ Error

Multiple Conditions

```
SQL  
CREATE TABLE Employee(  
    age INT CHECK(age BETWEEN 18 AND 60)  
);
```

Valid

```
25  
40
```

Invalid

10
70

Interview Questions

Real World Uses

```
SQL
CHECK(age >= 18)
CHECK(salary > 0)
CHECK(gender IN ('M', 'F'))
CHECK(status IN ('ACTIVE', 'INACTIVE'))
```

5. DEFAULT Constraint

What is DEFAULT?

Assigns a default value if no value is provided.

Example

```
SQL
CREATE TABLE Employee(
  emp_id INT,
  status VARCHAR(20)
  DEFAULT 'ACTIVE'
);
```

Insert

```
SQL
INSERT INTO Employee(emp_id)
VALUES(1);
```

Stored Data

emp_id	status
1	ACTIVE

Date Example

SQL

```
CREATE TABLE Orders(  
  order_id INT,  
  created_date DATE  
  DEFAULT CURRENT_DATE  
);
```

Insert

SQL

```
INSERT INTO Orders(order_id)  
VALUES(1001);
```

Date automatically inserted.

Interview Questions

Why Use DEFAULT?

Avoid inserting common values repeatedly.

Examples:

```
ACTIVE  
PENDING  
CURRENT_DATE  
CREATED_BY
```

NOT NULL (Very Important)

What is NOT NULL?

Column must have a value.

Example

SQL

```
CREATE TABLE Employee(  
  emp_id INT,  
  name VARCHAR(100) NOT NULL  
);
```

Valid

SQL

```
INSERT INTO Employee  
VALUES(1, 'John');
```

Invalid

```
SQL
INSERT INTO Employee
VALUES(1,NULL);
```

✗ Error

Frequently Asked Interview Questions

Q1. Difference Between Primary Key and Unique Key?

Feature	Primary Key	Unique Key
Duplicate	Not Allowed	Not Allowed
NULL	Not Allowed	Allowed
Count per Table	One	Multiple

Q2. Can Foreign Key Reference Unique Key?

✓ Yes

A Foreign Key can reference:

- Primary Key
- Unique Key

Q3. Can Primary Key Be NULL?

✗ No

Q4. Can Foreign Key Be NULL?

✓ Yes

Unless NOT NULL is specified.

Q5. Can a Table Have Multiple Unique Keys?

✓ Yes

Example:

```
SQL
email UNIQUE,
phone UNIQUE,
aadhaar UNIQUE
```

Q6. What Happens If Parent Record Is Deleted?

Depends on constraint options:

```
SQL
ON DELETE CASCADE
ON DELETE SET NULL
ON DELETE RESTRICT
```

Example:

```
SQL
FOREIGN KEY(dept_id)
REFERENCES Department(dept_id)
ON DELETE CASCADE
```

Deleting a department automatically deletes related employees.

Interview Shortcut

Constraint	Purpose
PRIMARY KEY	Unique + Not Null
FOREIGN KEY	Relationship Between Tables
UNIQUE	Prevent Duplicate Values
CHECK	Validate Data
DEFAULT	Assign Default Value
NOT NULL	Mandatory Value

Most Common Interview Question

"Explain the difference between Primary Key, Foreign Key, and Unique Key."

Constraint	Duplicate Allowed	NULL Allowed
PRIMARY KEY	✗ No	✗ No
FOREIGN KEY	✓ Yes	✓ Yes
UNIQUE KEY	✗ No	✓ Yes

This is one of the most frequently asked SQL theory questions for Java, Spring Boot, and Full Stack Developer interviews.

Indexes
 Clustered Index
 Non-Clustered Index
 Composite Index
 Explain Indexes

SQL Indexes - Complete Interview Guide

What is an Index?

An **Index** is a database object that improves the speed of data retrieval operations (**SELECT**) by creating a fast lookup structure.

Think of it like the index page of a book.

Without Index

Suppose Employee table has 10 lakh records.

```
SQL
SELECT *
FROM Employee
WHERE emp_id = 500000;
```

Database may scan every row until it finds the record.

1 → 2 → 3 → ... → 500000

This is called a **Full Table Scan**.

With Index

```
SQL
CREATE INDEX idx_emp_id
ON Employee(emp_id);
```

Now database can quickly locate the row.

Book Index Example

Java → Page 100
 SQL → Page 200
 Spring → Page 300

Instead of reading the whole book.

Why Use Indexes?

Faster Searches

```
SQL
SELECT *
FROM Employee
WHERE emp_id = 100;
```

Faster Sorting

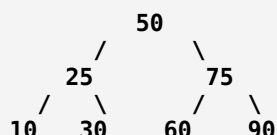
```
SQL
SELECT *
FROM Employee
ORDER BY salary;
```

Faster Joins

```
SQL
SELECT *
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id;
```

How Index Works Internally?

Most databases use a **B-Tree (Balanced Tree)** structure.



Searching becomes much faster.

Time Complexity

Without Index:

$O(n)$

With Index:

$O(\log n)$

Interviewers often ask this.

Create Index

```
SQL
CREATE INDEX idx_employee_name
ON Employee(name);
```

View Indexes

(MySQL)

```
SQL
SHOW INDEX
FROM Employee;
```

Drop Index

```
SQL
DROP INDEX idx_employee_name
ON Employee;
```

Clustered Index

What is Clustered Index?

A Clustered Index determines the **physical order of data in the table**.

Important Rule

Only ONE Clustered Index per table

Because data can be physically sorted only one way.

Example

Employee Table

emp_id	name
1	John
2	Alice
3	Bob

Primary Key:

```
SQL
CREATE TABLE Employee(
  emp_id INT PRIMARY KEY
);
```

Most databases automatically create a Clustered Index on the Primary Key.

Visualization

Without Clustered Index

```
3 Bob
1 John
2 Alice
```

With Clustered Index

```
1 John
2 Alice
3 Bob
```

Data stored in sorted order.

Interview Questions

Can a Table Have Multiple Clustered Indexes?

✗ No

Only one.

Why?

Because physical storage order can be only one.

Non-Clustered Index

What is Non-Clustered Index?

Stores index separately from table data.

Contains:

```
Indexed Value
+
Pointer to Actual Row
```

Example

```
SQL
CREATE INDEX idx_name
ON Employee(name);
```

Index Structure

```
Alice → Row 2
Bob   → Row 3
John  → Row 1
```

Actual data remains unchanged.

Visualization

Table

```
1 John
2 Alice
3 Bob
```

Index

```
Alice → Row 2
Bob → Row 3
John → Row 1
```

Interview Questions

How Many Non-Clustered Indexes?

☒ Multiple

```
SQL
CREATE INDEX idx_name ON Employee(name);
CREATE INDEX idx_email ON Employee(email);
CREATE INDEX idx_phone ON Employee(phone);
```

Clustered vs Non-Clustered Index

Feature	Clustered	Non-Clustered
Physical Data Sorted	Yes	No
Count Per Table	One	Many
Faster Range Queries	Yes	Good
Additional Storage	Less	More
Usually On	Primary Key	Frequently searched columns

Composite Index

What is Composite Index?

An index created on multiple columns.

Example

```
SQL
```

```
CREATE INDEX idx_emp_dept
ON Employee(name, dept_id);
```

Index contains:

```
(name, dept_id)
```

Why Composite Index?

Suppose query:

```
SQL
SELECT *
FROM Employee
WHERE name='John'
AND dept_id=101;
```

Composite Index is faster than two separate indexes.

Left-Most Prefix Rule

Most Important Interview Question.

Index:

```
SQL
CREATE INDEX idx_emp
ON Employee(name, dept_id);
```

Uses Index

```
SQL
WHERE name='John'
```

☒ Yes

```
SQL
WHERE name='John'
AND dept_id=101
```

☒ Yes

```
SQL
WHERE dept_id=101
```

☒ Usually No

Because first column (name) is missing.

Memory Trick

```
(name, dept_id)
```

Use:

name

name + dept_id

Not:

dept_id alone

When NOT to Create Indexes?

Avoid indexing:

Small Tables

10-20 records don't need indexes.

Frequently Updated Columns

SQL

```
UPDATE Employee
```

```
SET salary = ...
```

Each update requires index maintenance.

Low Cardinality Columns

Example:

Gender

M

F

Very few unique values.

Index is usually ineffective.

Advantages of Indexes

Faster SELECT

SQL

```
SELECT *
```

```
FROM Employee
```

```
WHERE email='abc@gmail.com';
```

Faster JOIN

Faster ORDER BY

Faster GROUP BY

Disadvantages of Indexes

More Storage

Indexes consume disk space.

Slower INSERT

```
SQL
INSERT INTO Employee ...
```

Index must be updated.

Slower UPDATE

Slower DELETE

Real-World Example

Login System

```
SQL
SELECT *
FROM Users
WHERE email='john@gmail.com';
```

Create index:

```
SQL
CREATE UNIQUE INDEX idx_email
ON Users(email);
```

Very common in production systems.

Most Asked Interview Questions

Q1. What is an Index?

A database object that improves query performance by enabling faster data retrieval.

Q2. Why Is Index Faster?

Uses B-Tree structure instead of scanning every row.

Q3. Clustered vs Non-Clustered?

Clustered	Non-Clustered
Data sorted physically	Separate structure
One only	Multiple
Faster for range queries	Faster for lookups

Q4. Can a Table Have Multiple Clustered Indexes?

✗ No

Only one.

Q5. Can a Table Have Multiple Non-Clustered Indexes?

✓ Yes

Q6. Which Columns Should Be Indexed?

Good candidates:

```
SQL
email
username
emp_id
customer_id
foreign keys
```

Frequently used in:

```
SQL
WHERE
JOIN
ORDER BY
GROUP BY
```

Q7. Why Not Create Index on Every Column?

Because:

```
More Storage
Slower INSERT
Slower UPDATE
Slower DELETE
```

Q8. What Is Composite Index?

Index on multiple columns.

```
SQL
```



```
CREATE INDEX idx_emp
ON Employee(name, dept_id);
```

Q9. Explain Left-Most Prefix Rule

For index:

```
SQL
(name, dept_id)
```

Works:

```
SQL
WHERE name=' John '
```

Works:

```
SQL
WHERE name=' John '
AND dept_id=101
```

Does not efficiently work:

```
SQL
WHERE dept_id=101
```

Interview Shortcut

Index Type	Description
Clustered Index	Sorts actual table data
Non-Clustered Index	Separate lookup structure
Composite Index	Index on multiple columns

One-Line Interview Answer

"An Index is a data structure (usually B-Tree) that improves SELECT query performance by reducing the number of rows scanned. Clustered indexes sort the actual data, non-clustered indexes store pointers to data, and composite indexes optimize searches on multiple columns."

ACID Properties
 Atomicity
 Consistency
 Isolation
 Durability

ACID Properties - Complete Interview Guide

ACID is a set of properties that ensures database transactions are processed reliably and maintain data integrity.

A **Transaction** is a group of SQL operations treated as a single unit of work.

Real-World Example: Bank Transfer

Suppose John transfers ₹1000 to Alice.

Initial Balances:

Account	Balance
John	₹5000
Alice	₹3000

Transaction:

```
SQL
BEGIN;

UPDATE Account
SET balance = balance - 1000
WHERE account_id = 1;

UPDATE Account
SET balance = balance + 1000
WHERE account_id = 2;

COMMIT;
```

After transaction:

Account	Balance
John	₹4000
Alice	₹4000

ACID properties ensure this transaction is safe.

A - Atomicity

What is Atomicity?

"All or Nothing"

Either all operations in a transaction succeed, or none of them do.

Example

Money deducted from John's account:

```
SQL
UPDATE Account
SET balance = balance - 1000
WHERE account_id = 1;
```

But before crediting Alice, server crashes.

Without Atomicity:

Account	Balance
John	₹4000
Alice	₹3000

₹1000 is lost.

✗ Incorrect

With Atomicity:

Database performs:

```
SQL
ROLLBACK;
```

Result:

Account	Balance
John	₹5000
Alice	₹3000

✓ Correct

Interview Answer

Atomicity ensures that either all operations of a transaction are completed successfully or all are rolled back.

C - Consistency

What is Consistency?

Database must move from one valid state to another valid state.

Rules, constraints, and relationships must always remain valid.

Example

Account balances:

```
John = 5000
Alice = 3000

Total = 8000
```

After transfer:

```
John = 4000
Alice = 4000

Total = 8000
```

Database remains consistent.

Constraint Example

```
SQL
CREATE TABLE Employee(
  age INT CHECK(age >= 18)
);
```

Attempt:

```
SQL
INSERT INTO Employee VALUES(10);
```

✗ Rejected

Because it violates consistency rules.

Interview Answer

Consistency ensures that a transaction does not violate database rules, constraints, or relationships.

I - Isolation

What is Isolation?

Multiple transactions should not interfere with each other.

Each transaction should behave as if it is running alone.

Example

Account Balance = ₹5000

Transaction T1

```
SQL
UPDATE Account
SET balance = balance - 1000;
```

Transaction T2

```
SQL
SELECT balance
FROM Account;
```

If T2 reads before T1 commits:

```
Balance = 4000
```

But T1 later rolls back.

Actual balance:

```
5000
```

This causes incorrect results.

Isolation prevents such issues.

Problems Solved by Isolation

Dirty Read

Reading uncommitted data.

Example

Transaction T1:

```
SQL
UPDATE Account
SET balance = 4000;
```

Not committed yet.

Transaction T2 reads:

```
4000
```

Then T1 rolls back.

Actual balance:

5000

T2 read invalid data.

Non-Repeatable Read

Same row gives different results.

Example:

Transaction T1:

```
SQL
SELECT balance;
```

Returns:

5000

Transaction T2 updates balance:

6000

Transaction T1 reads again:

6000

Different result.

Phantom Read

New rows appear during transaction.

Example:

```
SQL
SELECT *
FROM Employee
WHERE dept='IT';
```

Returns 10 employees.

Another transaction inserts a new IT employee.

Same query returns:

11 employees

A phantom row appeared.

Isolation Levels

Most asked interview topic.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	No	Possible	Possible
Repeatable Read	No	No	Possible
Serializable	No	No	No

Read Uncommitted

Fastest.

Allows dirty reads.

Lowest Isolation
Highest Performance

Read Committed

Default in many databases.

Prevents dirty reads.

Repeatable Read

Prevents:

- Dirty Reads
- Non-Repeatable Reads

Still allows phantom reads.

Serializable

Highest isolation level.

Transactions execute one after another.

Safest but slowest.

D - Durability

What is Durability?

Once a transaction is committed, data must survive permanently.

Even if:

- Server crashes
- Database restarts
- Power failure occurs

Example

Transaction committed:

```
SQL  
COMMIT;
```

Balance becomes:

```
John = 4000  
Alice = 4000
```

Immediately after commit:

```
Power Failure
```

After restart:

Balances should still be:

```
John = 4000  
Alice = 4000
```

How Durability Is Achieved?

Using:

- Transaction Logs
- Redo Logs
- Write-Ahead Logging (WAL)
- Disk Persistence

Interview Answer

Durability guarantees that once a transaction is committed, its changes are permanently stored and cannot be lost.

ACID Summary Table

Property	Meaning
Atomicity	All or Nothing
Consistency	Valid State to Valid State
Isolation	Transactions Don't Interfere
Durability	Committed Data Is Permanent

Interview Questions

Q1. What is a Transaction?

A transaction is a group of SQL operations executed as a single unit of work.

Example:

```
SQL
BEGIN;

UPDATE Account
SET balance=balance-1000
WHERE id=1;

UPDATE Account
SET balance=balance+1000
WHERE id=2;

COMMIT;
```

Q2. Which ACID Property Uses COMMIT and ROLLBACK?

✓ Atomicity

```
SQL
COMMIT;
ROLLBACK;
```

Q3. Which ACID Property Prevents Dirty Reads?

✓ Isolation

Q4. Which ACID Property Ensures Data Is Saved After Crash?

✓ Durability

Q5. Which ACID Property Ensures Constraints Are Followed?

✓ Consistency

Q6. What Happens If a Server Crashes Before COMMIT?

```
SQL
BEGIN;

UPDATE Account
SET balance=4000;
```

Server crashes.

Transaction is rolled back.

Because it was not committed.

This demonstrates:

✓ Atomicity

Q7. What Happens If Server Crashes After COMMIT?

Changes remain saved.

This demonstrates:

✓ Durability

Banking Interview Scenario

Question:

"Money was deducted from Account A but not credited to Account B due to server failure. Which ACID property handles this?"

Answer:

✓ Atomicity

The transaction will roll back and restore the original balances.

Quick Memory Trick

```
A = All or Nothing
C = Correct Data
I = Independent Transactions
D = Data Stays Forever
```

One-Line Interview Answer

"ACID properties ensure reliable database transactions: Atomicity guarantees all-or-nothing execution, Consistency maintains valid data, Isolation prevents transaction interference, and Durability ensures committed data survives failures."

Normalization
1NF
2NF
3NF
BCNF

Database Normalization - Complete Interview Guide

What is Normalization?

Normalization is the process of organizing data in a database to:

- Reduce data redundancy (duplicate data)
- Improve data integrity
- Avoid update, insert, and delete anomalies
- Make database design efficient

Why Do We Need Normalization?

Unnormalized Table

emp_id	emp_name	dept_id	dept_name
1	John	101	IT
2	Alice	101	IT
3	Bob	102	HR

Problem:

IT repeated multiple times
HR repeated multiple times

This wastes storage and creates maintenance issues.

Problems Without Normalization

1. Update Anomaly

Department changes:

IT → Information Technology

Must update every employee record.

If one row is missed:

IT
Information Technology

Inconsistent data.

2. Insert Anomaly

Want to add a new department:

Finance

But no employee exists yet.

Cannot insert department separately.

3. Delete Anomaly

Delete last employee from HR.

Bob deleted

HR department information is lost.

1NF (First Normal Form)

Rule

- Each column must contain atomic (single) values.
- No repeating groups.
- No multi-valued columns.

Not in 1NF

emp_id	name	skills
1	John	Java, SQL

emp_id	name	skills
2	Alice	Angular, React

Problem:

skills contains multiple values

Convert to 1NF

emp_id	name	skill
1	John	Java
1	John	SQL
2	Alice	Angular
2	Alice	React

Now each cell contains one value.

✓ 1NF achieved

Interview Definition

1NF ensures each column contains atomic values and eliminates repeating groups.

2NF (Second Normal Form)

Rule

Table must:

- Be in 1NF
- Remove Partial Dependency

What is Partial Dependency?

A non-key column depends on only part of a composite key.

Example

Student Course Table

student_id	course_id	student_name	course_name
1	101	John	Java
1	102	John	SQL
2	101	Alice	Java

Primary Key:

(student_id, course_id)

Problems:

student_name depends only on student_id
course_name depends only on course_id

Not on entire key.

This is Partial Dependency.

Convert to 2NF

Student Table

student_id	student_name
1	John
2	Alice

Course Table

course_id	course_name
101	Java
102	SQL

Student_Course Table

student_id	course_id
1	101
1	102
2	101

Now all columns depend on full key.

✅ 2NF achieved

Interview Definition

2NF removes partial dependency. Every non-key attribute must depend on the entire primary key.

3NF (Third Normal Form)

Rule

Table must:

- Be in 2NF
- Remove Transitive Dependency

What is Transitive Dependency?

A non-key column depends on another non-key column.

Example

emp_id	emp_name	dept_id	dept_name
1	John	101	IT
2	Alice	101	IT

Primary Key:

emp_id

Dependency:

emp_id → dept_id
dept_id → dept_name

Therefore:

emp_id → dept_name

This is Transitive Dependency.

Convert to 3NF

Employee Table

emp_id	emp_name	dept_id
1	John	101
2	Alice	101

Department Table

dept_id	dept_name
101	IT
102	HR

Now:

dept_name stored only once

✅ 3NF achieved

Interview Definition

3NF removes transitive dependencies. Non-key columns should depend only on the primary key.

BCNF (Boyce-Codd Normal Form)

Rule

A stronger version of 3NF.

Every determinant must be a candidate key.

Common Interview Example

Professor Subject Table

professor	subject
A	Java
A	SQL
B	Java

Assume:

Professor determines Subject

Subject determines Professor

Both relationships exist.

Even though table may satisfy 3NF, it may violate BCNF.

BCNF Rule

Every determinant should be a candidate key.

Simplified Interview Answer

BCNF = Advanced version of 3NF

Used when:

Multiple Candidate Keys
Complex Relationships

Normalization Progression

Unnormalized Form

emp_id	name	skills
1	John	Java,SQL

1NF

emp_id	name	skill
1	John	Java
1	John	SQL

2NF

Separate:

Student
Course
Student_Course

Remove partial dependency.

3NF

Separate:

Employee Department

Remove transitive dependency.

BCNF

Remove advanced dependency issues involving candidate keys.

Most Asked Interview Questions

Q1. What is Normalization?

Process of organizing data to reduce redundancy and improve integrity.

Q2. Why Normalize a Database?

To avoid:

- Duplicate data
- Update anomalies
- Insert anomalies
- Delete anomalies

Q3. What is 1NF?

Atomic values only.

Q4. What is 2NF?

Remove partial dependency.

Q5. What is 3NF?

Remove transitive dependency.

Q6. What is BCNF?

Every determinant must be a candidate key.

Q7. Difference Between 2NF and 3NF?

2NF	3NF
Removes Partial Dependency	Removes Transitive Dependency
Depends on Composite Keys	Depends on Non-Key Columns

Q8. Which Normal Form Is Most Common in Industry?

✓ 3NF

Most OLTP systems are designed up to 3NF.

Q9. What Is Denormalization?

Opposite of normalization.

Combines tables to improve read performance.

Example:

Employee
Department

Merged into one table.

Used in reporting systems and data warehouses.

Quick Memory Trick

1NF → Atomic Values

2NF → No Partial Dependency

3NF → No Transitive Dependency

BCNF → Every Determinant is Candidate Key

One-Line Interview Answer

"Normalization is the process of organizing database tables to eliminate redundancy and anomalies. 1NF ensures atomic values, 2NF removes partial dependency, 3NF removes transitive dependency, and BCNF is a stricter version of 3NF where every determinant must be a candidate key."

Views
Simple View
Materialized View

SQL Views - Complete Interview Guide

What is a View?

A **View** is a virtual table based on the result of a SQL query.

A view does not usually store data itself; it stores the SQL query and fetches data from underlying tables when accessed.

Why Use Views?

Security

Hide sensitive columns.

Example:

```
SQL
Employee
-----
emp_id
name
salary
email
```

HR should see salary.

Normal employees should not.

Create a view:

```
SQL
CREATE VIEW Employee_Public AS
SELECT emp_id,
       name,
       email
FROM Employee;
```

Users access the view instead of the actual table.

Simplify Complex Queries

Instead of writing:

```
SQL
SELECT e.emp_id,
       e.name,
       d.dept_name
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id;
```

every time, create a view.

```
SQL
CREATE VIEW EmployeeDetails AS
SELECT e.emp_id,
       e.name,
       d.dept_name
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id;
```

Now simply:

```
SQL
SELECT *
FROM EmployeeDetails;
```

Simple View

What is a Simple View?

A view based on a single table.

Example

Employee Table

emp_id	name	salary
1	John	50000
2	Alice	70000

Create View

```
SQL
CREATE VIEW EmployeeView AS
SELECT emp_id,
       name
FROM Employee;
```

Use View

```
SQL
SELECT *
FROM EmployeeView;
```

Output:

emp_id	name
1	John
2	Alice

Update Through View

Possible if:

- Single table
- No aggregate functions
- No GROUP BY
- No DISTINCT

Example:

```
SQL
UPDATE EmployeeView
SET name='Johnny'
WHERE emp_id=1;
```

Updates actual Employee table.

Interview Question

Is Data Stored in Simple View?

✗ No

Only query definition is stored.

Data comes from base table.

Complex View

(Not always asked, but useful)

Example

```
SQL
CREATE VIEW DepartmentSalary AS
SELECT dept_id,
       AVG(salary) avg_salary
FROM Employee
GROUP BY dept_id;
```

Contains:

- GROUP BY
- Aggregate functions

Generally not updatable.

Materialized View

What is a Materialized View?

A Materialized View physically stores the query result.

Unlike a normal view, data is stored on disk.

Simple View

```
View
→ Executes query every time
```

Materialized View

```
Query Result
→ Stored physically
```

Example

```
SQL
CREATE MATERIALIZED VIEW EmployeeSummary AS
SELECT dept_id,
       COUNT(*) total_employees,
       AVG(salary) avg_salary
FROM Employee
GROUP BY dept_id;
```

Data is stored.

Why Use Materialized Views?

Suppose:

```
SQL
SELECT dept_id,
       COUNT(*),
       AVG(salary)
FROM Employee
GROUP BY dept_id;
```

takes:

10 seconds

Run thousands of times daily.

Instead:

```
SQL
SELECT *
FROM EmployeeSummary;
```

takes milliseconds.

Materialized View Refresh

Since data is stored, it can become outdated.

Refresh Manually

```
SQL
REFRESH MATERIALIZED VIEW EmployeeSummary;
```

Updates stored data.

Example

Employee added:

```
SQL
INSERT INTO Employee
VALUES (...);
```

Materialized view still contains old data until refreshed.

Interview Question

Why Is Materialized View Faster?

Because:

Query already executed
Result already stored

No need to recompute.

View vs Materialized View

Feature	View	Materialized View
Stores Query	Yes	Yes
Stores Data	No	Yes
Execution Time	Every Access	Precomputed
Speed	Slower	Faster
Storage Needed	Minimal	Additional Storage
Data Always Latest	Yes	May Need Refresh

Real-World Examples

Banking

View

```
SQL
CREATE VIEW CustomerInfo AS
SELECT customer_id,
       name,
       email
FROM Customer;
```

Hide sensitive columns.

Reporting System

Materialized View

```
SQL
CREATE MATERIALIZED VIEW MonthlySales AS
SELECT month,
       SUM(amount)
FROM Sales
GROUP BY month;
```

Used for dashboards.

Most Asked Interview Questions

Q1. What is a View?

A virtual table based on a SQL query.

Q2. Does a View Store Data?

Simple View

✗ No

Stores only SQL definition.

Materialized View

✓ Yes

Stores actual query results.

Q3. Why Use Views?

- Security
 - Simplicity
 - Reusability
 - Abstraction
-

Q4. Can We Update a View?

✓ Yes, if:

- Single table
- No aggregate functions
- No GROUP BY
- No DISTINCT

Example:

```
SQL
UPDATE EmployeeView
SET name='John'
WHERE emp_id=1;
```

Q5. Difference Between View and Table?

Table	View
Stores Data	Usually Doesn't
Physical Object	Virtual Object
Faster Access	Depends on Query

Q6. Difference Between View and Materialized View?

View	Materialized View
Virtual	Physical
No Stored Data	Stores Data
Always Fresh	Needs Refresh
Slower	Faster

Q7. Why Materialized View Is Faster?

Because:

Query Result Already Stored

No repeated calculation.

Q8. Drawback of Materialized View?

- Requires storage
- Needs refresh
- Data may become stale

Frequently Asked Production Scenario

Dashboard Query Is Slow

Query:

```
SQL
SELECT dept_id,
COUNT(*),
AVG(salary)
```

```
FROM Employee
GROUP BY dept_id;
```

Runs every minute.

Solution

Create:

```
SQL
CREATE MATERIALIZED VIEW EmployeeSummary ...
```

Dashboard reads from Materialized View.

Performance improves significantly.

Interview Shortcut

Object	Purpose
View	Virtual Table
Simple View	Single Table View
Materialized View	Stored Query Result

Quick Memory Trick

```
View
=
Query Stored

Materialized View
=
Query Result Stored
```

One-Line Interview Answer

"A View is a virtual table created from a SQL query and typically does not store data, whereas a Materialized View physically stores the query result, making reads faster but requiring periodic refreshes."

Explain
Stored Procedures
Functions
Triggers

Stored Procedures, Functions, and Triggers – Complete Interview Guide

These are database objects used to implement business logic inside the database.

1. Stored Procedure

What is a Stored Procedure?

A **Stored Procedure** is a precompiled collection of SQL statements stored in the database and executed when called.

Think of it as a method/function inside the database.

Why Use Stored Procedures?

- Reuse SQL code
- Improve performance
- Reduce network traffic
- Centralize business logic

Example

Create Procedure

```
SQL
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT *
    FROM Employee;
END;
```

Execute Procedure

```
SQL
CALL GetEmployees();
```

or in SQL Server:

```
SQL
EXEC GetEmployees;
```

Procedure with Parameters

Input Parameter

```
SQL
```

```
CREATE PROCEDURE GetEmployeeById(
  IN empId INT
)
BEGIN
  SELECT *
  FROM Employee
  WHERE emp_id = empId;
END;
```

Call:

```
SQL
CALL GetEmployeeById(1);
```

Real-World Example

Bank Transfer

```
SQL
CREATE PROCEDURE TransferMoney(
  IN fromAcc INT,
  IN toAcc INT,
  IN amount DECIMAL(10,2)
)
BEGIN
  UPDATE Account
  SET balance = balance - amount
  WHERE id = fromAcc;

  UPDATE Account
  SET balance = balance + amount
  WHERE id = toAcc;
END;
```

Interview Questions

Can a Procedure Return Multiple Values?

☒ Yes

- Result sets
- Output parameters

Can Procedure Contain Transactions?

☒ Yes

```
SQL
BEGIN
COMMIT
ROLLBACK
```

supported.

2. Function

What is a Function?

A Function is a database object that:

- Accepts parameters
- Returns exactly one value

Used inside SQL statements.

Example

```
SQL
CREATE FUNCTION GetBonus(
    salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
BEGIN
    RETURN salary * 0.10;
END;
```

Usage

```
SQL
SELECT name,
       GetBonus(salary)
FROM Employee;
```

Output

name	bonus
John	5000
Alice	7000

Real-World Example

Calculate Tax

```
SQL
CREATE FUNCTION CalculateTax(
    amount DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
BEGIN
```

```
RETURN amount * 0.18;
END;
```

Use:

```
SQL
SELECT CalculateTax(1000);
```

Result:

```
180
```

Interview Questions

Can Function Modify Data?

Generally  No

Functions should return values and avoid changing database state.

Can Function Be Used in SELECT?

 Yes

```
SQL
SELECT GetBonus(salary)
FROM Employee;
```

Stored Procedure vs Function

This is one of the most asked interview questions.

Feature	Procedure	Function
Returns Value	Optional	Mandatory
Multiple Values	Yes	No
Can Be Called in SELECT	No	Yes
Can Modify Data	Yes	Usually No
Supports Transactions	Yes	Limited

Example

Procedure

```
SQL
CALL GetEmployees();
```


Function

```
SQL
SELECT GetBonus(50000);
```

3. Trigger

What is a Trigger?

A Trigger is a special database object that automatically executes when a specific event occurs.

Events:

```
INSERT
UPDATE
DELETE
```

Why Use Triggers?

- Audit logging
- Data validation
- Automatic calculations
- Change tracking

Trigger Example

Suppose we want to track salary updates.

Audit Table

```
SQL
CREATE TABLE Employee_Audit(
    emp_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    updated_on TIMESTAMP
);
```

Trigger

```
SQL
CREATE TRIGGER SalaryAudit
AFTER UPDATE
ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Audit(
        emp_id,
```

```

        old_salary,
        new_salary,
        updated_on
    )
VALUES(
    OLD.emp_id,
    OLD.salary,
    NEW.salary,
    NOW()
);

END;
```

Execution

```

SQL
UPDATE Employee
SET salary = 70000
WHERE emp_id = 1;
```

Trigger runs automatically.

Audit Table

emp_id	old_salary	new_salary
1	50000	70000

Types of Triggers

BEFORE Trigger

Runs before operation.

```

SQL
BEFORE INSERT
BEFORE UPDATE
BEFORE DELETE
```

Example

Validate salary.

```

SQL
BEFORE INSERT
```

Reject if salary is negative.

AFTER Trigger

Runs after operation succeeds.

SQL
 AFTER INSERT
 AFTER UPDATE
 AFTER DELETE

Used for:

- Logging
- Notifications
- Audit records

Real-World Examples

Banking

Audit all transactions.

INSERT into transaction table

Trigger logs details.

E-Commerce

After order insertion:

Reduce product stock

Automatically.

HR System

After salary update:

Store old salary

in audit table.

Trigger Lifecycle

INSERT
 ↓
 Trigger Fires

Most Asked Interview Questions

Q1. What is a Stored Procedure?

A precompiled collection of SQL statements stored in the database.

Q2. What is a Function?

A reusable database object that accepts parameters and returns a value.

Q3. What is a Trigger?

A database object that executes automatically when INSERT, UPDATE, or DELETE occurs.

Q4. Procedure vs Function?

Procedure	Function
May return value	Must return value
Cannot be used in SELECT	Can be used in SELECT
Can modify data	Usually should not

Q5. What Are OLD and NEW in Triggers?

Used to access row values.

SQL
OLD.salary

Value before update.

SQL
NEW.salary

Value after update.

Q6. Difference Between BEFORE and AFTER Trigger?

BEFORE	AFTER
Before operation	After operation
Validation	Logging/Auditing

Q7. When Should Triggers Be Avoided?

Because:

- Hidden logic
- Hard to debug
- Can affect performance

Many modern applications prefer handling business logic in the application layer (e.g., Java Spring Boot).

Quick Comparison

Feature	Procedure	Function	Trigger
Called Explicitly	Yes	Yes	No
Automatic Execution	No	No	Yes
Returns Value	Optional	Mandatory	No
Used in SELECT	No	Yes	No
Event Driven	No	No	Yes

Quick Memory Trick

```

Stored Procedure
=
Database Method

Function
=
Database Method That Returns Value

Trigger
=
Database Event Listener

```

One-Line Interview Answer

- **Stored Procedure:** A reusable set of SQL statements executed explicitly.
- **Function:** A reusable database object that accepts inputs and returns a value.

- **Trigger:** An automatically executed database object that fires on INSERT, UPDATE, or DELETE events.

Common Java Developer Interview Question

"When would you use a Trigger instead of a Stored Procedure?"

Answer:

Use a **Trigger** when logic must execute automatically on database events (audit logs, history tables). Use a **Stored Procedure** when logic should execute only when explicitly called by the application.

What is Normalization?
What is Denormalization?
Difference between View and Table?

1. What is Normalization?

Normalization is the process of organizing data in a database to reduce data redundancy (duplicate data) and improve data integrity.

Goals

- Remove duplicate data
- Reduce storage usage
- Maintain consistency
- Avoid anomalies (Insert, Update, Delete)

Example Before Normalization

emp_id	emp_name	dept_id	dept_name
1	John	101	IT
2	Alice	101	IT
3	Bob	102	HR

Problem:

IT repeated multiple times
HR repeated multiple times

After Normalization

Employee

emp_id	emp_name	dept_id
1	John	101
2	Alice	101
3	Bob	102

Department

dept_id	dept_name
101	IT
102	HR

Now department information is stored only once.

- ✓ Less redundancy
- ✓ Better consistency

Interview Answer

Normalization is the process of organizing database tables to minimize data redundancy and improve data integrity by dividing data into related tables.

2. What is Denormalization?

Denormalization is the process of combining normalized tables to improve read performance.

It is the opposite of normalization.

Example

Normalized Tables:

Employee

emp_id	emp_name	dept_id
1	John	101

Department

dept_id	dept_name
101	IT

To fetch employee details:

```
SQL
SELECT e.emp_name,
       d.dept_name
FROM Employee e
JOIN Department d
ON e.dept_id=d.dept_id;
```

Requires JOIN.

Denormalized Table

emp_id	emp_name	dept_id	dept_name
1	John	101	IT

No JOIN required.

Queries become faster.

Advantages of Denormalization

- Faster SELECT queries
- Fewer JOINS
- Better reporting performance
- Better analytics performance

Disadvantages

- Duplicate data
- More storage
- Update complexity
- Risk of inconsistency

Where Used?

Normalization

Banking Systems
ERP Systems
OLTP Applications

Denormalization

Normalization vs Denormalization

Feature	Normalization	Denormalization
Data Redundancy	Low	High
Storage	Less	More
Read Performance	Slower	Faster
Write Performance	Better Consistency	More Updates
JOINS	More	Less
Data Integrity	High	Lower

Interview Question

Which is used in Banking Systems?

✓ Normalization

Reason:

Data consistency is more important than speed.

Which is used in Reporting Systems?

✓ Denormalization

Reason:

Read performance is more important.

3. Difference Between View and Table

This is a very common interview question.

Table

A table is a physical database object that stores actual data.

Example

SQL

```
CREATE TABLE Employee(
  emp_id INT,
  name VARCHAR(100)
);
```

Data is physically stored.

View

A view is a virtual table created from a SQL query.

Example

```
SQL
CREATE VIEW EmployeeView AS
SELECT emp_id,
       name
FROM Employee;
```

Usually stores only the query definition.

Example

Table Data

emp_id	name	salary
1	John	50000
2	Alice	70000

View

```
SQL
CREATE VIEW PublicEmployee AS
SELECT emp_id,
       name
FROM Employee;
```

Output:

emp_id	name
1	John
2	Alice

Salary is hidden.

Why Use Views?

Security

Hide sensitive columns.

```
SQL
salary
password
credit_card
```

Simplicity

Complex query:

```
SQL
SELECT e.name,
       d.dept_name
FROM Employee e
JOIN Department d
ON e.dept_id=d.dept_id;
```

Can be wrapped inside a view.

Reusability

Use the same logic repeatedly.

Table vs View

Feature	Table	View
Stores Data	Yes	Usually No
Physical Object	Yes	Virtual Object
Contains Rows	Yes	Derived from Query
Storage Required	Yes	Minimal
Can Have Indexes	Yes	Generally No (except materialized views)
Security	Less	Better
Based On Query	No	Yes

Interview Questions

Q1. What is Normalization?

Answer:

Normalization organizes data into multiple related tables to reduce redundancy and improve integrity.

Q2. What is Denormalization?

Answer:

Denormalization combines tables and introduces redundancy to improve read performance.

Q3. Why is Normalization Important?

To avoid:

- Update Anomaly
 - Insert Anomaly
 - Delete Anomaly
-

Q4. Why Use Denormalization?

To improve:

- Reporting
 - Analytics
 - Dashboard performance
-

Q5. What is a View?

A virtual table created from a SQL query.

Q6. Does a View Store Data?

Normal View

 No

Stores only query definition.

Materialized View

 Yes

Stores actual query results.

Q7. Difference Between View and Table?

Table

Stores actual data

View

Stores SQL query

Quick Memory Trick

```
Normalization
=
Reduce Redundancy

Denormalization
=
Improve Read Performance

Table
=
Stores Data

View
=
Stores Query
```

One-Line Interview Answers

Normalization: Organizing data into related tables to eliminate redundancy and maintain consistency.

Denormalization: Combining tables and introducing controlled redundancy to improve query performance.

View vs Table: A table physically stores data, whereas a view is a virtual table that displays data from one or more underlying tables through a query.

What is AUTO_INCREMENT?
 Locks
 Shared Lock
 Exclusive Lock
 Explain Query
 How does MySQL handle transactions?

1. What is AUTO_INCREMENT?

AUTO_INCREMENT is a MySQL feature that automatically generates unique sequential values for a column, usually a Primary Key.

Example

```
SQL
CREATE TABLE Employee(
  emp_id INT AUTO INCREMENT PRIMARY KEY,
  name VARCHAR(100)
);
```

Insert data:

```
SQL
INSERT INTO Employee(name)
VALUES ('John');

INSERT INTO Employee(name)
VALUES ('Alice');
```

Output:

emp_id	name
1	John
2	Alice

MySQL automatically generates IDs.

Why Use AUTO_INCREMENT?

Without it:

```
SQL
INSERT INTO Employee
VALUES(1, 'John');

INSERT INTO Employee
VALUES(2, 'Alice');
```

You must manually manage IDs.

With AUTO_INCREMENT:

```
SQL
INSERT INTO Employee(name)
VALUES('John');
```

MySQL handles it.

Interview Questions

What happens if row 2 is deleted?

```
SQL
DELETE FROM Employee
WHERE emp_id=2;
```

Next insert:

```
SQL
INSERT INTO Employee(name)
VALUES( 'Bob' );
```

Result:

```
emp_id = 3
```

MySQL does not reuse deleted values by default.

Reset AUTO_INCREMENT

```
SQL
ALTER TABLE Employee
AUTO_INCREMENT = 1000;
```

Next inserted record gets:

```
1000
```

2. Locks in MySQL

Locks prevent multiple transactions from corrupting data.

Imagine two users updating the same bank account simultaneously.

Without locks:

```
User A updates balance
User B updates balance
```

Data inconsistency may occur.

Locks solve this problem.

Shared Lock (Read Lock)

What is Shared Lock?

Allows multiple transactions to read data simultaneously.

But nobody can modify the locked data.

Example

Transaction T1

```
SQL
SELECT *
FROM Employee
WHERE emp_id = 1
LOCK IN SHARE MODE;
```

or in newer MySQL:

```
SQL
SELECT *
FROM Employee
WHERE emp_id = 1
FOR SHARE;
```

What Happens?

Transaction T2

```
SQL
SELECT *
FROM Employee;
```

✓ Allowed

Transaction T3

```
SQL
UPDATE Employee
SET salary=50000
WHERE emp_id=1;
```

✗ Waits until lock is released.

Shared Lock Summary

```
Read ✓
Read ✓
Write ✗
```

Exclusive Lock (Write Lock)

What is Exclusive Lock?

Allows one transaction to read and modify data.

Other transactions cannot read or write depending on isolation level.

Example

```
SQL
SELECT *
FROM Employee
WHERE emp_id = 1
FOR UPDATE;
```

What Happens?

Transaction T1:

```
SQL
SELECT *
FROM Employee
WHERE emp_id=1
FOR UPDATE;
```

Gets Exclusive Lock.

Transaction T2:

```
SQL
UPDATE Employee
SET salary=70000
WHERE emp_id=1;
```

✗ Waits

Transaction T3:

```
SQL
DELETE FROM Employee
WHERE emp_id=1;
```

✗ Waits

Exclusive Lock Summary

```
Read Limited
Write ✗
Delete ✗
Update ✗
```

Shared vs Exclusive Lock

Feature	Shared Lock	Exclusive Lock
Read Allowed	Yes	Usually Yes for owner

Feature	Shared Lock	Exclusive Lock
Write Allowed	No	Only owner
Multiple Users	Yes	No
Used For	Reading	Updating

3. Explain Query (EXPLAIN)

One of the most important interview topics.

What is EXPLAIN?

EXPLAIN shows how MySQL executes a query.

It helps identify:

- Table scans
- Index usage
- Join order
- Query cost

Example

```
SQL
EXPLAIN
SELECT *
FROM Employee
WHERE emp_id = 100;
```

Output:

id	table	type	key
1	Employee	const	PRIMARY

Why Use EXPLAIN?

Suppose query is slow:

```
SQL
SELECT *
FROM Employee
WHERE email='abc@gmail.com';
```

Use:

```
SQL
EXPLAIN
SELECT *
FROM Employee
WHERE email='abc@gmail.com';
```

to see whether index is being used.

Important EXPLAIN Types

const

Best performance.

Primary Key Lookup

ref

Uses non-unique index.

Good.

range

Uses range scan.

```
SQL
WHERE salary BETWEEN 10000 AND 50000
```

Good.

index

Entire index scanned.

Moderate.

ALL

Worst case.

Full Table Scan.

Reads every row

Usually indicates missing index.

Interview Question

Which EXPLAIN type is worst?

✗ ALL

Because MySQL scans the entire table.

4. How Does MySQL Handle Transactions?

MySQL uses **ACID properties** and transaction logs to ensure data consistency.

Transactions are supported by the InnoDB storage engine.

Transaction Lifecycle

Start Transaction

```
SQL
START TRANSACTION;
```

or

```
SQL
BEGIN;
```

Execute Queries

```
SQL
UPDATE Account
SET balance = balance - 1000
WHERE id = 1;

UPDATE Account
SET balance = balance + 1000
WHERE id = 2;
```

Commit

```
SQL
COMMIT;
```

Changes become permanent.

Rollback

```
SQL
ROLLBACK;
```

Undo all changes since transaction started.

Banking Example

Before Transaction

Account	Balance
A	5000
B	3000

Transaction

```
SQL
BEGIN;

UPDATE Account
SET balance = balance - 1000
WHERE id = 1;

UPDATE Account
SET balance = balance + 1000
WHERE id = 2;

COMMIT;
```

After Transaction

Account	Balance
A	4000
B	4000

What Happens Internally?

1. Transaction Starts

```
SQL
BEGIN;
```

MySQL creates transaction context.

2. Changes Written to Redo Log

Before writing to disk:

```
Redo Log
```

stores changes.

3. Locks Applied

Rows being modified receive locks.

```
SQL
SELECT ... FOR UPDATE
```

or implicit row locks.

4. COMMIT

```
SQL
COMMIT;
```

Changes become permanent.

5. Crash Recovery

If server crashes:

MySQL uses:

```
Redo Log
Undo Log
```

to recover.

This ensures:

☒ Durability

Interview Questions

Q1. What is AUTO_INCREMENT?

Automatically generates unique numeric values for a column.

Q2. Difference Between Shared and Exclusive Lock?

Shared	Exclusive
Read Only	Read + Write
Multiple Users	Single Owner
Used for Reading	Used for Updates

Q3. What is EXPLAIN?

Used to analyze query execution plans and optimize performance.

Q4. Which EXPLAIN Type Is Worst?

ALL

Full Table Scan.

Q5. Which MySQL Engine Supports Transactions?

✓ InnoDB

Q6. What Happens During COMMIT?

- Changes written permanently
- Locks released
- Transaction completed

Q7. What Happens During ROLLBACK?

- All uncommitted changes undone
- Database restored to previous state

Quick Memory Trick

AUTO_INCREMENT
=
Automatic Primary Key

Shared Lock
=

Many Readers
 Exclusive Lock
 =
 Single Writer

 EXPLAIN
 =
 Query Execution Plan

 Transaction
 =
 BEGIN → SQL → COMMIT/ROLLBACK

One-Line Interview Answer

MySQL transactions are handled by InnoDB using ACID properties, row-level locking, redo logs, undo logs, and COMMIT/ROLLBACK mechanisms to ensure data consistency and durability.

PostgreSQL vs MySQL?
 What is MVCC?
 What is JSONB?
 Advantages of PostgreSQL?
 What is Full Text Search?
 Explain PostgreSQL indexing.

1. PostgreSQL vs MySQL

This is one of the most common interview questions.

Feature	PostgreSQL	MySQL
Type	Object-Relational DB	Relational DB
ACID Compliance	Strong	Good (InnoDB)
Performance	Better for complex queries	Better for simple reads
JSON Support	JSON + JSONB	JSON
Full Text Search	Built-in, powerful	Available but less powerful
MVCC	Advanced	Available via InnoDB
Stored Procedures	Advanced	Basic
Extensibility	High	Moderate
Index Types	More options	Fewer options
Community Usage	Enterprise, Analytics	Web Applications

When to Use MySQL?

Good for:

- Web Applications

- E-commerce
- CRUD-heavy systems
- Simpler architectures

Examples:

- WordPress
 - Magento
 - Laravel Applications
-

When to Use PostgreSQL?

Good for:

- Banking systems
 - Financial systems
 - Analytics
 - GIS applications
 - Complex reporting
-

Interview Answer

MySQL is generally preferred for simple, high-speed web applications, while PostgreSQL is preferred for complex queries, advanced features, data integrity, and enterprise applications.

2. What is MVCC?

MVCC = Multi-Version Concurrency Control

A mechanism that allows multiple users to access the same data simultaneously without blocking each other.

One of PostgreSQL's strongest features.

Problem Without MVCC

User A

SQL

```
UPDATE Employee
SET salary = 70000
WHERE id = 1;
```

User B

```
SQL
SELECT *
FROM Employee
WHERE id = 1;
```

Without MVCC:

User B waits

Performance decreases.

With MVCC

PostgreSQL keeps multiple versions of rows.

Example:

Old Version

Salary = 50000

New Version

Salary = 70000

User A sees:

70000

User B sees:

50000

Both work simultaneously.

Benefits

No Read Locks

Readers don't block writers.

No Write Locks on Reads

Writers don't block readers.

Better Concurrency

Thousands of users can access data simultaneously.

Interview Question

Why is PostgreSQL good for concurrent systems?

Because of MVCC.

3. What is JSONB?

JSONB = Binary JSON

PostgreSQL supports:

```
SQL
JSON
JSONB
```

JSON

Stores raw JSON text.

```
JSON
{
  "name": "John",
  "age": 25
}
```

Every query parses JSON again.

JSONB

Stores JSON in binary format.

```
Preprocessed
Indexed
Optimized
```

Faster querying.

Example

```
SQL
CREATE TABLE users(
  id INT,
  profile JSONB
);
```

Insert:

```
SQL
INSERT INTO users
VALUES(
1,
'{
  "name": "John",
  "city": "Mumbai"
}'
);
```

Query JSONB

```
SQL
SELECT profile->>'name'
FROM users;
```

Result:

```
John
```

JSONB Index

```
SQL
CREATE INDEX idx_profile
ON users
USING GIN(profile);
```

Very fast searching.

JSON vs JSONB

Feature	JSON	JSONB
Storage	Text	Binary
Query Speed	Slower	Faster
Index Support	Limited	Excellent
Recommended	Rarely	Mostly

Interview Answer

JSONB stores JSON data in a binary format that supports indexing and faster querying compared to regular JSON.

4. Advantages of PostgreSQL

1. Strong ACID Compliance

Excellent transaction support.

```
SQL
BEGIN;
COMMIT;
ROLLBACK;
```

2. Advanced MVCC

High concurrency.

Readers and writers don't block each other.

3. JSONB Support

Store semi-structured data efficiently.

Example:

```
JSON
{
  "address": {
    "city": "Mumbai"
  }
}
```

4. Advanced Indexing

Supports:

- B-Tree
- Hash
- GIN
- GiST
- BRIN

5. Full Text Search

Built-in search engine capabilities.

6. Window Functions

Excellent analytical query support.

```
SQL
ROW_NUMBER()
RANK()
DENSE_RANK()
```

7. Extensions

Popular extensions include:

- PostGIS
- pg_stat_statements
- pgcrypto

Interview Answer

PostgreSQL offers advanced indexing, MVCC, JSONB support, powerful querying capabilities, strong ACID compliance, and excellent support for enterprise applications.

5. What is Full Text Search?

Allows searching text like a search engine.

Instead of:

```
SQL
SELECT *
FROM articles
WHERE title LIKE '%java%';
```

PostgreSQL provides optimized searching.

Example

Table:

```
SQL
CREATE TABLE articles(
  id INT,
  content TEXT
);
```

Full Text Search Query

```
SQL
SELECT *
FROM articles
WHERE to_tsvector(content)
@@ to_tsquery('java');
```

Meaning

to_tsvector

Converts text into searchable tokens.

to_tsquery

Converts search term.

Example

Content:

Java Spring Boot Tutorial

Search:

```
SQL
SELECT *
FROM articles
WHERE to_tsvector(content)
@@ to_tsquery('spring');
```

Result found quickly.

Why Better Than LIKE?

LIKE

```
SQL
LIKE '%spring%'
```

Scans large amounts of text.

Full Text Search

Uses indexes.

Much faster.

Real-World Usage

- Product search
 - Blog search
 - Document search
 - Knowledge bases
-

6. PostgreSQL Indexing

Indexes improve query performance.

PostgreSQL supports several index types.

B-Tree Index

Default index type.

Best for:

```
SQL
=
<
>
BETWEEN
ORDER BY
```

Example

```
SQL
CREATE INDEX idx_emp_name
ON employee(name);
```

Hash Index

Best for:

```
SQL
=
```


queries.

Example:

```
SQL
CREATE INDEX idx_email
ON employee
USING HASH(email);
```

GIN Index

Very important interview topic.

Used for:

- JSONB
- Arrays
- Full Text Search

Example:

```
SQL
CREATE INDEX idx_profile
ON users
USING GIN(profile);
```

GiST Index

Used for:

- Geospatial data
- Range queries

Popular with spatial applications.

BRIN Index

Used for huge tables.

Example:

```
100 Million Rows
```

Consumes very little storage.

PostgreSQL Index Types Summary

Index Type	Best For
B-Tree	Equality, Range Queries
Hash	Equality
GIN	JSONB, Full Text Search
GiST	Geospatial Data
BRIN	Very Large Tables

Most Asked Interview Questions

Q1. PostgreSQL vs MySQL?

Answer: PostgreSQL provides more advanced features such as MVCC, JSONB, advanced indexing, and Full Text Search, while MySQL is often simpler and widely used for web applications.

Q2. What is MVCC?

Multi-Version Concurrency Control allows readers and writers to work simultaneously without blocking each other.

Q3. What is JSONB?

A binary JSON format in PostgreSQL that supports indexing and faster queries.

Q4. Why JSONB Instead of JSON?

- Faster
- Indexable
- Better performance

Q5. What Index Is Used For JSONB?

SQL
GIN Index

Q6. Which Index Is Used For Full Text Search?

SQL
GIN Index

Q7. Which Is PostgreSQL's Default Index?

SQL
B-Tree

Q8. Why Is PostgreSQL Popular In Enterprise Applications?

Because of:

- Strong ACID compliance
- MVCC
- Advanced indexing
- JSONB
- Full Text Search
- High concurrency

Quick Memory Trick

```

MVCC
=
Many Versions

JSONB
=
Binary JSON

GIN
=
JSONB + Full Text Search

B-Tree
=
Default Index

PostgreSQL
=
Advanced Features + Enterprise Ready

```

One-Line Interview Answer

PostgreSQL is an advanced open-source database known for MVCC, JSONB support, Full Text Search, powerful indexing options, and strong transactional consistency, making it well-suited for enterprise and data-intensive applications.

What is MongoDB?
Basic syntax and fuctions/commands in Mongo DB

[Collection vs Document?](#)
[MongoDB vs SQL?](#)
[What is Replication?](#)
[What is Sharding?](#)
[What is Aggregation?](#)
[Embedded vs Referenced Documents?](#)
[Show more](#)[Show less](#)

MongoDB - Complete Interview Guide

What is MongoDB?

MongoDB is a **NoSQL Document Database** that stores data in **JSON-like BSON documents** instead of tables and rows.

Unlike SQL databases:

```

MySQL/PostgreSQL
→ Tables → Rows → Columns

MongoDB
→ Collections → Documents → Fields
    
```

MongoDB is schema-flexible, meaning documents in the same collection can have different structures.

Collection vs Document

SQL

Employee Table

id	name	salary
1	John	50000

MongoDB

Employee Collection

```

JSON
{
  "_id": 1,
  "name": "John",
  "salary": 50000
}
    
```

Collection

Equivalent to a SQL table.

```
Employee Collection
```

Contains multiple documents.

Document

Equivalent to a row in SQL.

```
JSON
{
  "_id": 1,
  "name": "John"
}
```

Interview Answer

SQL	MongoDB
Table	Collection
Row	Document
Column	Field

Basic MongoDB Commands

Create Database

```
JavaScript
use companyDB
```

Creates or switches database.

Show Databases

```
JavaScript
show dbs
```

Show Collections

```
JavaScript
show collections
```

Insert Document

insertOne()

```
JavaScript
db.employee.insertOne({
  name: "John",
  salary: 50000
})
```

insertMany()

```
JavaScript
db.employee.insertMany([
  {
    name: "John",
    salary: 50000
  },
  {
    name: "Alice",
    salary: 70000
  }
])
```

Find Documents

All Documents

```
JavaScript
db.employee.find()
```

Equivalent SQL:

```
SQL
SELECT *
FROM employee;
```

Pretty Output

```
JavaScript
db.employee.find().pretty()
```

Find One

```
JavaScript
db.employee.findOne()
```

Returns first matching document.

Filtering

Salary > 50000

```
JavaScript
db.employee.find({
  salary: { $gt: 50000 }
})
```

Equivalent SQL:

```
SQL
SELECT *
FROM employee
WHERE salary > 50000;
```

Operators

MongoDB	Meaning
\$gt	Greater Than
\$lt	Less Than
\$gte	Greater Than Equal
\$lte	Less Than Equal
\$eq	Equal
\$ne	Not Equal
\$in	IN

Example

```
JavaScript
db.employee.find({
  salary: {
    $gte: 50000,
    $lte: 80000
  }
})
```

Update Documents

updateOne()

```
JavaScript
db.employee.updateOne(
  {name: "John"},
  {$set: {salary: 60000}}
)
```

updateMany()

```
JavaScript
db.employee.updateMany(
  {},
  {$set: {status: "ACTIVE"}}
)
```

Delete Documents

deleteOne()

```
JavaScript
db.employee.deleteOne(
  {name: "John"}
)
```

deleteMany()

```
JavaScript
db.employee.deleteMany(
  {status: "INACTIVE"}
)
```

Sorting

```
JavaScript
db.employee.find()
.sort({salary: -1})
```

Descending.

Ascending:

```
JavaScript
.sort({salary: 1})
```


Limit

```
JavaScript
db.employee.find()
.limit(5)
```

Equivalent:

```
SQL
LIMIT 5
```

Count Documents

```
JavaScript
db.employee.countDocuments()
```

Equivalent:

```
SQL
SELECT COUNT(*)
FROM employee;
```

MongoDB vs SQL

Feature	MongoDB	SQL
Database Type	NoSQL	Relational
Storage	Documents	Tables
Schema	Flexible	Fixed
Joins	Limited	Strong
Scaling	Horizontal	Vertical
Transactions	Supported	Strong
Query Language	BSON/JSON	SQL

When To Use MongoDB?

Good for:

- Social Media
- E-commerce Catalogs
- Real-time Applications
- Content Management
- Microservices

What is Replication?

Replication means maintaining multiple copies of data across servers.

Purpose:

High Availability
Fault Tolerance
Disaster Recovery

Replica Set

MongoDB uses Replica Sets.

Primary Node

Handles Writes

Secondary Nodes

Copy Data From Primary

Example

```

Primary
|
-----
|           |
Secondary1  Secondary2
  
```

If primary crashes:

Secondary becomes Primary

Automatically.

Interview Answer

Replication keeps multiple copies of data on different servers for high availability and failover.

What is Sharding?

Sharding is MongoDB's horizontal scaling mechanism.

Data is split across multiple servers.

Without Sharding

Server 1
1 TB Data

Problem:

Storage Limit
Performance Issue

With Sharding

Shard1 → Customer 1-100000
Shard2 → Customer 100001-200000
Shard3 → Customer 200001-300000

Data distributed.

Components

Shard

Stores actual data.

Config Server

Stores metadata.

Mongos

Query Router.

Interview Answer

Sharding distributes data across multiple servers to improve scalability and performance.

What is Aggregation?

Aggregation performs calculations on documents.

Equivalent of:

SQL
GROUP BY
COUNT

SUM
AVG

in SQL.

Example

Employee Collection

JSON

```
{
  "name": "John",
  "department": "IT",
  "salary": 50000
}
```

Total Salary By Department

JavaScript

```
db.employee.aggregate([
  {
    $group: {
      _id: "$department",
      totalSalary: {
        $sum: "$salary"
      }
    }
  }
])
```

Count Employees By Department

JavaScript

```
db.employee.aggregate([
  {
    $group: {
      _id: "$department",
      count: {
        $sum: 1
      }
    }
  }
])
```

Common Aggregation Stages

Stage	Purpose
\$match	WHERE
\$group	GROUP BY
\$sort	ORDER BY

Stage	Purpose
\$project	SELECT Columns
\$limit	LIMIT

SQL Comparison

SQL:

```
SQL
SELECT department,
      AVG(salary)
FROM employee
GROUP BY department;
```

MongoDB:

```
JavaScript
db.employee.aggregate([
{
  $group:{
    _id:"$department",
    avgSalary:{
      $avg:"$salary"
    }
  }
}
])
```

Embedded vs Referenced Documents

Very common MongoDB interview question.

Embedded Documents

Store related data inside same document.

```
JSON
{
  "_id":1,
  "name":"John",
  "address":{
    "city":"Mumbai",
    "pincode":"400001"
  }
}
```

Advantages

- Faster reads

- No joins
- Simpler queries

Use When

One-to-One
Small One-to-Many

relationships.

Referenced Documents

Store related data separately.

Employee

```
JSON
{
  "_id": 1,
  "name": "John",
  "deptId": 101
}
```

Department

```
JSON
{
  "_id": 101,
  "deptName": "IT"
}
```

Advantages

- Less duplication
- Easier updates
- Better for large relationships

Use When

Many-to-Many
Large One-to-Many

relationships.

Embedded vs Referenced

Feature	Embedded	Referenced
Performance	Faster Reads	More Queries
Duplication	Higher	Lower
Joins Needed	No	Yes (\$lookup)
Best For	Small Related Data	Large Relationships

Most Asked MongoDB Interview Questions

Q1. What is MongoDB?

A NoSQL document database that stores data as BSON documents.

Q2. Collection vs Document?

Collection = Table
Document = Row

Q3. MongoDB vs SQL?

MongoDB is schema-flexible and document-based; SQL databases are table-based and schema-driven.

Q4. What is Replication?

Maintaining multiple copies of data for high availability.

Q5. What is Sharding?

Distributing data across multiple servers for horizontal scaling.

Q6. What is Aggregation?

Framework used for grouping, filtering, and calculating data.

Q7. Embedded vs Referenced Documents?

Embedded stores related data inside a document; Referenced stores data in separate collections and links them.

Q8. What is \$lookup?

MongoDB's equivalent of JOIN.

JavaScript

```
db.employee.aggregate([
{
  $lookup:{
    from:"department",
    localField:"deptId",
    foreignField:"_id",
    as:"department"
  }
}]
])
```

Quick Memory Trick

Collection = Table

Document = Row

Replication = High Availability

Sharding = Horizontal Scaling

Aggregation = GROUP BY

Embedded = Store Together

Referenced = Store Separately

One-Line Interview Answer

MongoDB is a NoSQL document database that stores data as BSON documents, supports flexible schemas, replication for high availability, sharding for horizontal scaling, and aggregation pipelines for complex data processing.